

Efficient Event-Boundary Indexing and Memory Consolidation for Long- Form Video

Mohd Amir Suhel

Master of Science in Artificial Intelligence



School of Computer Science and Electronic Engineering

Faculty of Engineering and Physical Sciences

University of Surrey

Guildford, Surrey, GU2 7XH, UK

September 2026

Supervised by: Dr. Xiatian Zhu

© Mohd Amir Suhel 2026

DECLARATION OF ORIGINALITY

I confirm that the project dissertation I am submitting is entirely my own work and that any material used from other sources has been clearly identified and properly acknowledged and referenced. In submitting this final version of my report to the JISC anti-plagiarism software resource, I confirm that my work does not contravene the university regulations on plagiarism as described in the Student Handbook. In so doing I also acknowledge that I may be held to account for any particular instances of uncited work detected by the JISC anti-plagiarism software, or as may be found by the project examiner or project organiser. I also understand that if an allegation of plagiarism is upheld via an Academic Misconduct Hearing, then I may forfeit any credit for this module or a more severe penalty may be agreed.

EFFICIENT EVENT-BOUNDARY INDEXING AND MEMORY CONSOLIDATION FOR LONG-FORM VIDEO

Mohd Amir Suhel

Author Signature: Amir Suhel Date: 01/09/2026

Supervisor's name: Dr. Xiatian Zhu

WORD COUNT

Number of Pages: 80

Number of Words: 23,434

ABSTRACT

Continuous video is recorded at a scale nobody can watch. Wearable cameras, home monitoring and robots produce footage measured in days, and its value lies in being able to ask questions of it afterwards. No model can attend to days of video at once, so such systems build an external memory as the video arrives and retrieve from it at query time. How that memory is structured decides what can later be found, and at what cost.

WorldMM [1], the closest comparable published system, maintains episodic, semantic and visual memory banks over egocentric video. Two of its choices are inherited rather than designed, it cuts the video on a fixed 30-second grid taken from the dataset's file layout, splitting actions and storing unchanging stretches repeatedly, and its semantic store re-writes the whole accumulated state at every update.

This dissertation replaces both. Event-boundary indexing cuts where the visual signal changes, scoring differenced colour histograms and grayscale thumbnails against a rolling median and median absolute deviation, with no vision-language model in the loop. An offline consolidation pass then merges near-duplicate facts and stores each once with a validity interval.

Over 43 hours from one EgoLife participant, against a matched fixed-grid baseline, chunk count fell from 5,156 to 2,092, a 59.4% reduction, with 75.2% of boundaries produced by detection. Consolidation compresses the fact set by 1.58x, interval encoding shrinks the semantic store on disk by 101.4x and 128.4x, and the database is 51% smaller. Accuracy on 38 matched benchmark questions was identical at 36.8%.

Cutting on content rather than the clock halves the memory at no measured cost to retrieval. The gains are narrower than those ratios suggest:, most of the storage win is a serialisation choice, cold start improves only 1.15x to 1.22x, and one merge in nine joins facts about different people.

TABLE OF CONTENTS

Efficient Event-Boundary Indexing and Memory Consolidation for Long-Form Video.....	i
Declaration of originality	ii
Word Count	iii
Abstract.....	iv
Table of Contents	v
List of Figures	1
List of Tables.....	2
1 Introduction.....	3
1.1 Background and Context.....	3
1.1.1 Why long video, and why it is hard	3
1.1.2 Why memory is the pivotal component	4
1.1.3 How existing systems build such a memory, and where they fall short.....	5
1.1.4 What this dissertation proposes.....	5
1.2 Scope and Objectives	6
1.2.1 Objectives	6
1.2.2 Tasks undertaken to fulfil them.....	6
1.2.3 Scope and delimitation.....	6
1.2.4 Ethical and data-protection considerations	7
1.3 Achievements.....	8
1.4 Overview of Dissertation	8
2 Background Theory and Literature Review.....	10
2.1 Long-form video understanding and the memory problem	10
2.2 WorldMM: the direct baseline	11
2.3 Graph-structured retrieval over extracted text	12
2.4 Temporal segmentation: shots, scenes and event boundaries.....	12
2.5 Memory consolidation in biology and in artificial systems	13
2.6 EgoLife and EgoLifeQA as the evaluation setting.....	14
2.7 How systems of this kind are evaluated.....	15
2.8 Summary and the gap addressed	15
3 Event-Boundary Indexing.....	17
3.1 The baseline system and the experimental platform	17
3.1.1 How the baseline builds its memory	17
3.1.2 What this work changes, and what it holds fixed	18
3.1.3 Experimental design and the controls applied	19
3.1.4 Attribution.....	20

3.1.5	Corpus, hardware and models.....	21
3.2	Establishing a sound baseline	21
3.3	Boundary detection method	22
3.3.1	Design constraints.....	22
3.3.2	Features.....	23
3.3.3	Dissimilarity and adaptive thresholding	23
3.3.4	Why not a learned boundary model	24
3.3.5	Computational cost	24
3.4	From boundaries to chunks	24
3.5	Configuration and sensitivity analysis	25
3.5.1	Final configuration.....	25
3.5.2	Threshold sensitivity.....	26
3.5.3	Configurations rejected during development.....	27
3.6	Indexing results	28
3.6.1	Chunk statistics	28
3.6.2	Boundary provenance	29
3.6.3	How far the visual proxy can be trusted	31
3.7	Summary	32
4	Memory Consolidation and System Evaluation	33
4.1	What WorldMM's consolidation actually does	33
4.1.1	Why the facts cannot merge.....	34
4.2	The interval-encoded semantic store.....	35
4.2.1	Where the storage reduction comes from	35
4.3	The sleep-cycle consolidation algorithm	37
4.4	Consolidation results.....	39
4.4.1	Storage	39
4.4.2	Compression, and the orthogonality result	40
4.4.3	Cost	41
4.4.4	Merge quality: what the compression actually costs.....	42
4.4.5	Node period: a confound that had to be removed	43
4.5	Database size.....	44
4.6	Load and indexing latency	45
4.6.1	Load, model-free.....	45
4.6.2	Indexing with the embedding model: a negative result	46
4.6.3	Cold-start latency: composing the two measurements.....	47
4.7	Extraction cost.....	48
4.8	Retrieval accuracy.....	48

4.8.1 How much this evidence is worth	49
4.8.2 Per-category outcomes.....	50
4.9 Threats to validity	51
4.10 Summary	52
5 Conclusion	54
5.1 Evaluation	54
5.2 Reflection on the conduct of the project	57
5.3 Future Work	58
References.....	60
Appendix A — Origin of the components used	63
Appendix B — Experimental configuration	64
B.2 Consolidation configuration.....	65
B.3 Evaluation configuration	66
B.4 Environment	67
Appendix C — A worked example: one span through the whole pipeline	68
Appendix D — Glossary and notation.....	71

LIST OF FIGURES

Figure	Caption	Page
1-1	Questions from the benchmark, and the recording that answers them	3
1-2	How a memory-based video question-answering system works	4
2-1	What the three memory banks hold	11
2-2	What a fixed grid does to an activity stream	12
3-1	The baseline memory pipeline	17
3-2	The same pipeline with this dissertation's two changes	18
3-3	Sensitivity of the segmentation to the z-score threshold	26
3-4	Chunk-length distribution and empirical CDF, both conditions	29
3-5	Boundary provenance by day	30
3-6	A ten-minute window under both segmentations	31
4-1	Growth of WorldMM's inherited semantic store	34
4-2	Triples written per semantic node under each encoding	37
4-3	Consolidated store on disk: identical content, two encodings	39
4-4	Consolidation compression against similarity threshold	40
4-5	Database size by component	44
4-6	Load time and indexing time	46
4-7	EgoLifeQA retrieval outcomes	49

LIST OF TABLES

Table	Caption	Page
2-1	Where the contribution sits among related systems	16
3-1	What is held constant across the two arms	19
3-2	Confounds identified, and how each was controlled	20
3-3	Recovery over 14,496 extraction calls	22
3-4	Detection configuration	25-26
3-5	Threshold sweep, uncapped and capped	26-27
3-6	Configurations evaluated during method development	27
3-7	Chunk statistics, both conditions	28
3-8	Boundary provenance by day	30
4-1	WorldMM's measured consolidation behaviour	33
4-2	The storage reduction decomposed	36
4-3	Consolidation threshold sweep	40
4-4	Consolidation cost	41
4-5	Merge-quality audit of the consolidated stores	43
4-6	Database size by component	44
4-7	Load latency, model-free	45
4-8	Indexing latency with the embedding model, over 5 query points	46
4-9	Cold-start latency, composed	47
4-10	Measured extraction cost	48
4-11	Matched-pair contingency	49
4-12	Outcomes by question category	50
5-1	Objectives against measured outcomes	54-55
C-1	Episodic triples extracted from the worked example	68
C-2	Semantic triples at the first node	69
C-3	The consolidated interval record	69

1 INTRODUCTION

This dissertation is an experimental and software engineering project in computer vision and multimodal machine learning. It concerns how an artificial agent should organise a memory of video that runs for days rather than minutes, and specifically how such a system should decide where one remembered unit ends and the next begins, and what it should keep. The work is empirical, a reference system was reproduced, two of its mechanisms were replaced, and the consequences of both replacements were measured on the same 43 hours of egocentric footage under matched conditions.

Section 1.1 sets out why long video needs a memory at all, how existing systems build one, and where the two mechanisms replaced here fall short. Section 1.2 states the objectives and the limits of the study, Section 1.3 summarises what was achieved, and Section 1.4 describes the shape of the rest of the document..

1.1 Background and Context

1.1.1 Why long video, and why it is hard

Video is the largest and fastest-growing category of recorded data, and an increasing share of it is not produced to be watched. A wearable camera, a home monitoring installation or a service robot records continuously, generating footage measured in days and weeks. Nobody reviews it. Its value is realised only when a question is put to it afterwards: “where did I leave my keys?”, “did the delivery arrive at the front or the back door?”, “who was the last person to use the kitchen?”.

Answering such a question is difficult for a reason that has little to do with recognising objects or actions, both of which modern vision-language models do well. The difficulty is one of scale and of distance in time. Figure 1-1 makes the point with three questions taken from the benchmark used in this dissertation, plotted against the moments that actually contain their answers.

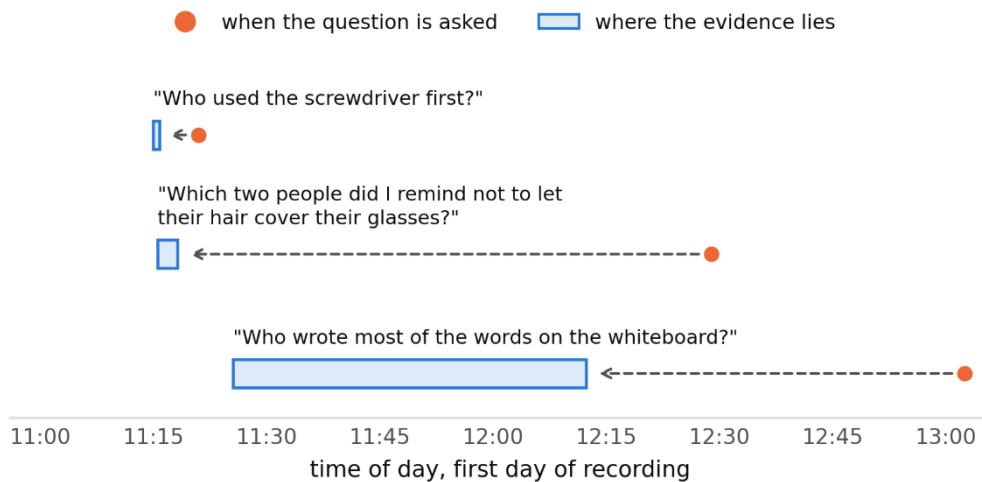


Figure 1-1 — Three questions from the EgoLifeQA benchmark [2], with the point at which each

is asked and the span of recording that contains the evidence. The gap ranges from six minutes to over an hour and a half, within a recording that runs for days.

Three properties follow from that picture and shape everything after it. The evidence is *far behind* the question, so answering means searching history rather than the present. The evidence is *sparse*, occupying seconds or minutes within hours of recording that are irrelevant to the question. And the questions are *not known in advance*, so the recording cannot be pre-processed for a specific query.

1.1.2 Why memory is the pivotal component

The obvious approach, giving the model the video and asking, does not survive contact with these properties. A vision-language model samples frames into a token sequence, and the cost of attending over that sequence grows quadratically with its length while the vision encoder cost grows linearly with the number of frames. A clip of a few minutes fits; forty-three hours does not, at any sampling rate that preserves the detail the questions depend on. Sampling more sparsely to fit trades away precisely the evidence being sought.

The response the field has converged on is to build an **external memory**. The stream is divided into units as it arrives, each unit is described by a captioner or a vision-language model, and the descriptions are indexed so that a small number of them can be retrieved when a question is asked. Figure 1-2 shows the shape of this.

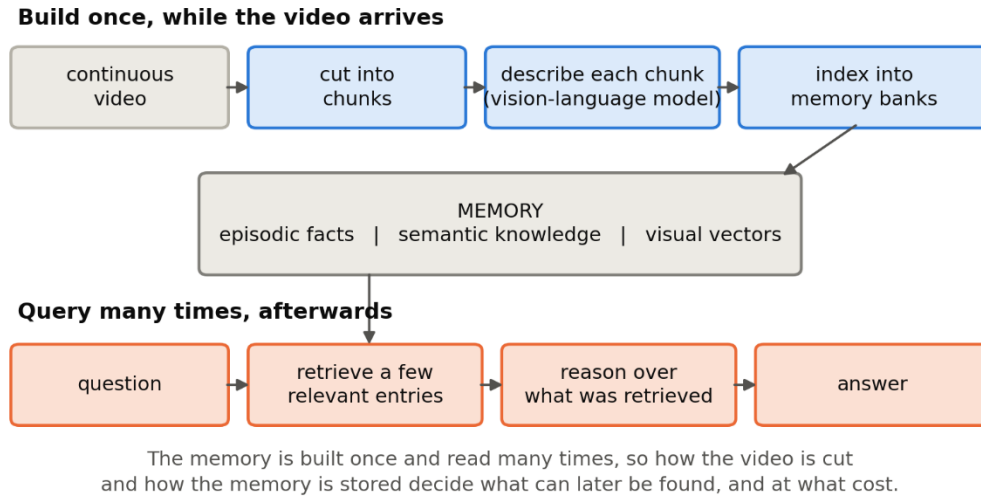


Figure 1-2 — *The two phases of a memory-based video question-answering system. The memory is built once, as the video arrives, and read many times afterwards.*

The asymmetry in that figure is the reason this dissertation exists. The memory is constructed once and consulted repeatedly, so decisions taken during construction are paid for once but constrain every later query. Two of those decisions are the subject of this work: how the stream is divided into units, and how the resulting knowledge is stored.

1.1.3 How existing systems build such a memory, and where they fall short

Several systems now implement this architecture for long video, and Chapter 2 reviews them in detail. The reference point for this dissertation is WorldMM [1], a multimodal memory agent presented as a CVPR 2026 Highlight, which is the most directly comparable published system for continuous egocentric video. It maintains three memory banks: an **episodic** store of what happened, held as subject-relation-object facts extracted per unit; a **semantic** store of generalised knowledge, updated periodically; and a **visual** store of one embedding vector per unit.

Almost every system of this kind, WorldMM included, divides the stream **uniformly**, into windows of fixed duration. Uniform division is trivial to implement, gives predictable memory growth, and requires no analysis of the content. It also cuts without regard to what is happening, with two consequences: an action that spans a boundary is split across two records, and a long stretch in which nothing changes is stored as many near-identical records. Neither failure is visible in the memory itself. Both surface later as retrieval errors, and as storage that grows in proportion to elapsed time rather than to the amount of distinct experience.

The second shortcoming concerns what happens to the semantic store over time. WorldMM's is described as consolidating, but Section 4.1 measures its behaviour on this project's own data and finds that it accumulates: the complete state is re-written at every update, so the volume written grows quadratically in the number of updates while the set of facts held barely changes. Nothing in it ever collapses a repeated observation into a single record.

Human memory does neither of these things. Cognitive research on event segmentation shows that people perceive continuous activity as a sequence of discrete events with boundaries at points of change, and that those boundaries are where encoding into memory is strongest [3]. Human memory also does not retain every episode indefinitely: during sleep, hippocampal traces are replayed and progressively integrated into neocortical semantic structure, a process that generalises across repeated experience rather than storing each instance [4], [5], [6]. Segment on change, and consolidate repetition offline. Both have direct engineering analogues, and neither is implemented by current video-memory systems.

1.1.4 What this dissertation proposes

This dissertation replaces the two inherited mechanisms and measures what changes. In place of the fixed grid, an **event-boundary indexing module** cuts the stream where the visual signal changes, detected by differencing cheap frame descriptors and scoring the result against a rolling robust threshold, with no vision-language model in the detection loop. In place of the accumulating semantic store, an **interval-encoded store** writes each fact once with the period over which it was observed, after an offline consolidation pass modelled on sleep-cycle replay has merged near-duplicates. Both are

evaluated against a matched baseline that reproduces the reference system's behaviour on the same data.

1.2 Scope and Objectives

1.2.1 Objectives

The following objectives are carried across from the approved project proposal unchanged. They were achievable as stated and are used in Section 5.1 as the basis for evaluation.

- **Objective 1:** Design an event-boundary aware indexing module to segment video streams based on semantic state changes rather than fixed temporal windows.
- **Objective 2:** Create an offline memory consolidation algorithm that mimics biological "sleep cycles" to collapse highly repetitive episodic event boundaries into singular, compressed semantic nodes.
- **Objective 3:** Evaluate the impact of boundary-aware indexing and consolidation on database compression ratios and retrieval speed.

1.2.2 Tasks undertaken to fulfil them

To make clear the scale of work behind the objectives: a CPU-only frame-feature extractor was implemented and run over 4,501 videos; a boundary detector was designed and swept across seven operating points in two cap regimes; the complete WorldMM memory pipeline was rebuilt from source for two experimental conditions, including approximately 41 GPU-hours of open information extraction on a shared cluster; an interval-encoded semantic store and a consolidation algorithm were written and swept across five similarity thresholds; a metrics harness producing five figure-grade datasets was written; and both conditions were evaluated on EgoLifeQA under identical retrieval settings. A defect in the inherited extraction code that had silently suppressed 13.2% of the baseline was diagnosed and fixed, which required the baseline arm to be rebuilt rather than reused. A test suite of 155 test functions across 10 files accompanies the code.

1.2.3 Scope and delimitation

The subject of this dissertation is **video indexing and memory consolidation**: everything that determines what is stored in the memory and in what form. Retrieval policy, what is fetched at query time and how, is deliberately held fixed throughout, so that any difference measured in Chapter 4 is attributable to the memory rather than to how it is queried.

Two components of the shared codebase were therefore used **unmodified**: the retrieval agent and the evaluation harness of Section 4.8, and a spatial memory bank that no experiment reported here exercises. Neither was written, modified or tuned as part of this work, and no result claimed in this dissertation depends on either beyond its role as a held constant. One further set of artifacts, a

previously generated run of the reference system over the same subject, appears in Section 4.1 as a behavioural reference and never as the baseline arm. Appendix A records the origin of every component used.

Three further limits on scope should be stated at the outset. The evaluation uses one of EgoLife's six subjects; and the retrieval evaluation is a bounded subset run text-only, which supports a statement that retrieval is not degraded and does not support a comparable benchmark score.

1.2.4 Ethical and data-protection considerations

The data used here is human-subject data of an unusually intimate kind — continuous first-person recording of six people living together for a week, including their conversation. Three points govern its use in this project.

No new recording was carried out. EgoLife [2] is a published dataset, collected by its authors under their own institutional consent framework with participants who agreed to be recorded and to have the recordings released for research, and it is used here solely under the terms of that release. The analysis is a secondary use of already-public data and introduces no additional risk of exposure beyond what the release already carries.

All processing was performed on University of Surrey infrastructure. Video, captions and transcripts were held on cluster storage and were not uploaded to any third-party service; every model used (Qwen3-VL-30B-A3B-Instruct, Qwen3-Embedding-4B, VLM2Vec-V2.0) runs locally on the cluster, so no participant content left university systems at any point.

No re-identification was attempted, and no result depends on identifying anyone. The participants appear in this dissertation only under the pseudonyms the dataset assigns (A1_JAKE, and the first names that occur inside the extracted triples, which are the dataset's own labels). The consolidated triples quoted in Section 4.4.5 are reproduced because merge quality cannot be audited without showing real examples; they describe mundane activity and carry no sensitive attribute.

One forward-looking concern deserves recording, because this dissertation makes memory of this kind cheaper to keep. Reducing the storage cost of a life-log by an order of magnitude lowers the practical barrier to retaining one indefinitely, and a memory that is cheap to keep is a memory that is more likely to be kept, and therefore more likely to be subpoenaed, breached or repurposed. The consolidation mechanism in Chapter 4 happens to offer a partial technical answer: because each fact carries *first_seen* and *last_seen* timestamps, the store supports principled time-bounded retention and selective forgetting in a way that a monolithic accumulated state does not: but this dissertation does not implement or evaluate such a policy, and Section 5.2 lists it as future work rather than claiming it as a contribution.

1.3 Achievements

- **Event-boundary indexing reduced the chunk count by 59.4%**, from 5,156 to 2,092, over 43 hours of video, raising mean chunk length from 27.9 s to 75.9 s. **75.2% of boundaries were produced by visual change detection** rather than by the maximum-length cap or by gaps in the recording, so the segmentation is driven by content and this is measured rather than asserted. Detection uses no vision-language model and runs on CPU.
- The operating threshold was **selected on a measured criterion independent of the evaluation**: $z = 4.0$ is the largest threshold at which detection, rather than the maximum-length cap, still places three quarters of the boundaries.
- **An interval-encoded semantic store reduced the consolidated memory on disk by 101.4x (fixed) and 128.4x (event)** relative to the reference format holding *identical content*. This is a comparison of two serialisations of the same facts, so it measures the encoding and not the consolidation algorithm; the algorithm's own compression is a more modest 1.58x (Section 4.4.2).
- **Consolidation eliminated all language-model calls**: zero, against approximately 8,757 for the fixed condition and 10,347 for the event condition under WorldMM's per-triple scheme, which at a measured 4 s per call would be roughly 9.7 hours of serialised generation.
- **The complete database is 51% smaller**, 23.22 MB against 47.51 MB, and the consolidated store **loads 5.7x to 11.2x faster**, composing to a **1.15x–1.22x cold-start** improvement once indexing is included.
- The storage figure is accounted for rather than merely observed. It follows from the reference format writing the whole accumulated state at every node while the interval store writes each fact once, and Section 4.2.1 reproduces both figures from separately measured quantities. The gain is a property of the serialisation format, not of a better compression algorithm.
- **Retrieval accuracy is unchanged**: on the 38 EgoLifeQA questions both configurations answered, both scored 36.8%, with symmetric disagreement and exact McNemar $p = 1.000$.

Three results run against the project's initial expectations and are reported in the main text rather than buried: there is **no indexing-time speedup** from the interval encoding once the embedding model is in the loop, and the end-to-end gain is 1.15x–1.22x rather than the order of magnitude the load figure alone implies; **consolidation is orthogonal to chunking** rather than compounding with it; and an audit of the consolidated store finds that **11.0%–14.3% of merges cross subjects**, a correctness defect that compression ratios alone would have hidden.

1.4 Overview of Dissertation

Chapter 2 reviews the literature the work sits in: long-form video memory architectures and WorldMM in particular; the graph retrieval method its episodic memory uses; the lineage of temporal

segmentation from shot-boundary detection to generic event boundaries; and the biological account of memory consolidation that motivates Objective 2. It closes by identifying the specific gap addressed.

Chapter 3 is the first technical chapter. It describes the experimental platform, the two conditions, and the engineering work required to make the baseline trustworthy before any comparison could be run: including the parser defect. It then presents the boundary-detection method, its configuration, the sensitivity analysis from which the operating point was chosen, and the indexing results that answer Objective 1.

Chapter 4 is the second technical chapter and carries the results. It begins by measuring what WorldMM's semantic consolidation actually does, which turns the premise of Objective 2 from an assertion into a measurement. It then presents the interval store and the sleep-cycle algorithm, and reports the full Objective 3 evaluation: compression, database size, load and index latency, extraction cost, and retrieval accuracy — including the results that do not favour the approach.

Chapter 5 evaluates the work against the three objectives in Section 1.2 and sets out what should follow from it.

2 BACKGROUND THEORY AND LITERATURE REVIEW

This chapter establishes the technical context in four movements. Section 2.1 sets out why long-form video forces an external memory and what the standard architecture looks like. Section 2.2 examines WorldMM in the detail required to understand precisely what this dissertation replaces, and Section 2.3 covers the graph-retrieval method its episodic memory depends on. Section 2.4 traces the segmentation literature that the Objective 1 method descends from, and Section 2.5 the memory-consolidation literature behind Objective 2. Section 2.6 describes the benchmark. Section 2.7 draws these together into the specific gap this work fills.

2.1 Long-form video understanding and the memory problem

Vision-language models process video by sampling frames into a token sequence [8], [9]. The cost of this is quadratic in sequence length for standard attention and linear in frames for the vision encoder, so there is a hard practical ceiling on how much video can be attended over at once. Sampling more sparsely trades away exactly the temporal detail that long-video questions depend on. For a corpus of forty-three hours, end-to-end attention is not an option at any sampling rate that preserves useful information.

The dominant response is retrieval-augmented generation adapted to video [10]. The stream is pre-processed into a durable index; at query time a retriever selects a small number of entries; and a generator answers from those entries alone. This is a direct descendant of text RAG, and it inherits text RAG's central sensitivity: retrieval quality is bounded by how the corpus was chunked. In text, chunking on semantic boundaries rather than on token counts is well understood to matter. In video, the equivalent question has received much less attention, and the equivalent of "chunk on token count" — cut every N seconds — remains the default.

Several systems have addressed the memory side of this without revisiting the segmentation. MovieChat [11] compresses a dense short-term token memory into a sparse long-term memory to extend the horizon. MA-LMM [12] maintains an explicit memory bank read by a querying transformer. VideoAgent [13] treats long-video understanding as an agentic loop in which a language model iteratively decides what to retrieve. In the language-model literature the analogous line runs through MemGPT [14], which manages a paged memory hierarchy, and generative agents [15], which maintain a memory stream with retrieval scored by recency, importance and relevance, and, notably for this work, a *reflection* step that periodically synthesises higher-level observations from accumulated low-level ones. That reflection step is the closest existing analogue to Objective 2, but it is implemented by prompting a language model to write summaries, which is exactly the per-item generation cost that Section 4.3 avoids.

Across all of these, the unit of memory is either a fixed window of frames or a single observation,

and the growth of stored state is monotonic in elapsed time.

2.2 WorldMM: the direct baseline

WorldMM [1] is the specific system this work modifies, and the details of its memory construction matter for the rest of the dissertation.

It maintains three memory banks over an egocentric stream:

- **Episodic memory** stores what happened, as open-information-extraction triples of the form *(subject, relation, object)* extracted by a vision-language model from each chunk's caption and transcript text: for example ["I", "hold", "phone"]. Entities are also extracted per chunk by a named-entity step, and the triples are assembled into a graph for retrieval.
- **Semantic memory** stores generalised knowledge, built by a periodic aggregation over the episodic stream: every 10 consecutive chunks, a semantic node is created that is supposed to consolidate what has been learned. On the 30-second grid, ten chunks is a five-minute node. Retrieval from this bank is itself graph-based: the implementation indexes triples into an entity graph and scores them by personalised PageRank, mirroring the episodic bank's HippoRAG retrieval.
- **Visual memory** stores a dense embedding per chunk: in the configuration used here, a 1536-dimensional VLM2Vec/VLM2Vec-V2.0 [31] vector per video file: supporting visual similarity retrieval. VLM2Vec-V2 extends the VLM2Vec contrastive embedding framework [16] to video.

Figure 2-1 shows what each bank holds, using real records from the reconstruction.

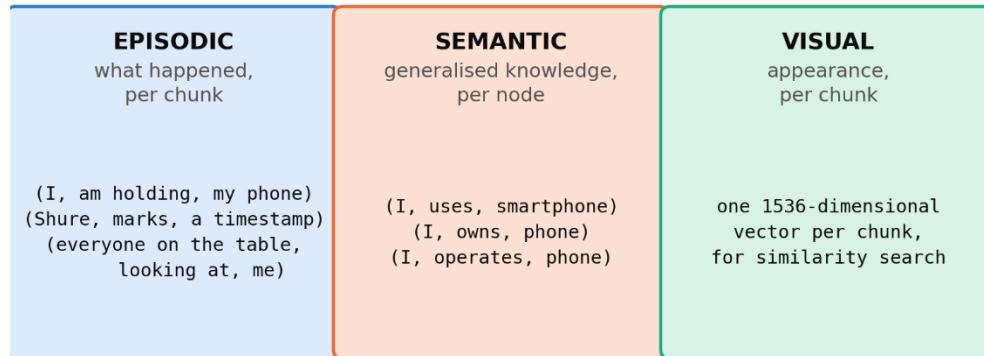


Figure 2-1 — *What each memory bank holds. The episodic and semantic examples are real records from the reconstruction described in Chapter 3; Appendix C traces one span of video through all three.*

Two properties of this construction are load-bearing for this dissertation.

First, **the chunk is the video file**. WorldMM does not choose a window size; it consumes the dataset's file layout. Because EgoLife ships as ~30 s slices, WorldMM's memory granularity is ~30 s. Nothing in the design would prevent a different granularity, and equally nothing in the design

considers what the granularity should be.

Second, the semantic "consolidation" step accumulates rather than consolidates. Section 4.1 reports the measurement, but the mechanism is worth stating here: each semantic node is written out holding the whole accumulated semantic state as of that point in time, rather than a delta or a compressed summary. The consequence is that the state carried by node n is re-serialised at node $n+1$, $n+2$ and so on, and the total volume written grows quadratically in the number of nodes even when the underlying set of facts barely changes.

2.3 Graph-structured retrieval over extracted text

WorldMM's episodic retrieval follows HippoRAG [7], which is itself explicitly neurobiologically motivated. HippoRAG builds a knowledge graph from open-information-extraction triples over a corpus, treats the graph as an analogue of the hippocampal index, and retrieves by running personalised PageRank from query-matched entity nodes: the artificial counterpart of pattern completion. Its relevance here is twofold. It explains why WorldMM's episodic memory is a triple store rather than a text store, which in turn is why the consolidation problem in this dissertation is a problem about *triples*. And it establishes the precedent of taking the hippocampal-neocortical division seriously as an architectural principle rather than as a metaphor, which is the stance Objective 2 takes.

Open information extraction itself has a long history [17], [18]. The relevant property for this work is that OpenIE relations are **unnormalised free text**. There is no controlled vocabulary; the extractor writes whatever relation phrase the sentence suggests. Section 4.1 shows the consequence on real data: the inherited semantic store holds 1,582 triples across 1,469 distinct predicates, so near-duplicate facts phrased slightly differently can never merge by string equality. This is the concrete, measurable target that Objective 2's consolidation addresses.

2.4 Temporal segmentation: shots, scenes and event boundaries

Before reviewing how the field cuts video, it is worth being precise about what a uniform grid costs. Figure 2-2 shows both failure modes on a stretch of ordinary activity.

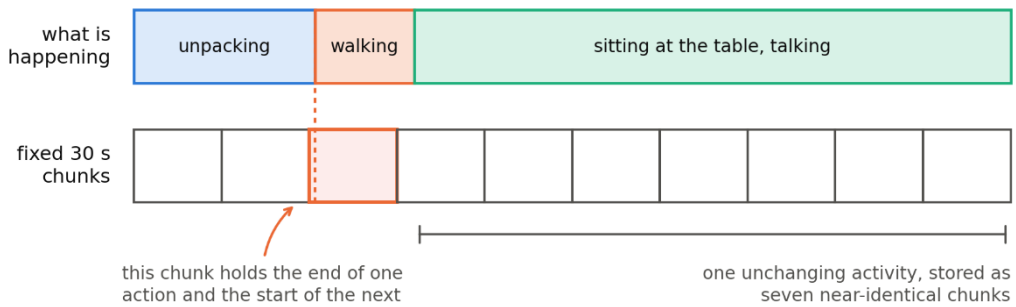


Figure 2-2 — *Schematic of the two costs of uniform segmentation. A boundary falls in the*

middle of an action, splitting its description across two records; and a long unchanging stretch is stored as many near-identical records. Neither is visible from inside the memory.

The idea of cutting video on content rather than on the clock is old, and Objective 1's method descends directly from it.

Shot boundary detection identifies cuts introduced by editing. The classical approach computes a low-level feature per frame, most commonly a colour histogram, differences it between adjacent frames, and thresholds the resulting dissimilarity signal [19]. Seven years of systematic TRECVID evaluation established that histogram-based differencing with an adaptive threshold is a strong and extremely cheap baseline for hard cuts, and that the difficulty lies almost entirely in gradual transitions [20]. PySceneDetect [21] remains the standard practical implementation of this family.

Scene boundary detection groups shots into semantically coherent scenes, typically by clustering shots on visual and audio similarity subject to temporal constraints [22].

Egocentric video has no shots. There are no edits, no camera changes and no gradual transitions; there is one continuous take from a moving head-mounted camera. What a histogram difference detects in this setting is not an edit but a change in what the wearer is looking at or doing: walking into a different room, turning from a screen to a person, standing up from a table. That is precisely the signal Objective 1 wants, and it is available without any semantic model.

Generic event boundary detection [23] formalises the target directly: given untrimmed video, predict the moments at which a human observer would say one event ended and another began, defined taxonomy-free. The GEBD benchmark showed that human boundary annotations agree well enough to be a learning target, and that strong performance is achievable. Its cognitive foundation is event segmentation theory [3]: perceivers maintain a working model of the current event and register a boundary when that model's predictions fail. A change in the visual statistics of the scene is the simplest available proxy for such a prediction failure.

The design choice this work makes is to use the *cheap classical* signal rather than a learned boundary model. The reasons are stated in full in Section 3.3, but the essential one is architectural: if boundary detection required a vision-language model, it would cost as much as the extraction it is meant to make cheaper, and the contribution would collapse.

2.5 Memory consolidation in biology and in artificial systems

Objective 2's "sleep cycle" is a specific analogy with a specific literature behind it, and the dissertation should be precise about which parts of it transfer.

Systems consolidation is the process by which memories initially dependent on the hippocampus become progressively independent of it and integrated into neocortical representations over time [4]. Complementary learning systems theory [5], updated in [6], explains why the brain would be built

this way: a fast-learning hippocampal system can encode individual episodes without catastrophic interference, while a slow-learning neocortical system extracts the structure common across episodes. Transfer between them occurs largely offline. Hippocampal place-cell sequences are replayed during slow-wave sleep [24], and sleep has been shown to be functionally necessary for the schema-building that turns repeated episodes into generalised knowledge [25]. The distinction between episodic and semantic memory itself is due to Tulving [26], and WorldMM's bank structure adopts it directly.

What transfers to an engineering setting is the *shape* of the process: an offline pass, run when the system is not serving queries, that takes a large set of individually recorded episodes and produces a smaller set of generalised records, with the originals' contribution preserved in aggregate rather than individually. What does not transfer, and should not be claimed, is any mechanistic equivalence — the algorithm in Section 4.3 is embedding-space clustering, not replay, and the analogy is a design motivation rather than a model of the brain.

Within machine learning, the same concerns appear as continual learning and rehearsal [27], where replaying stored examples counteracts catastrophic forgetting. That literature is concerned with preserving *model* parameters; the problem here is compressing a *database*. The nearest work in agent memory is the reflection mechanism of generative agents [15], which is offline, periodic and generalising, and therefore structurally the right analogue: but which spends a language-model call per synthesised memory. Section 4.3's design question is whether the generalisation step can be done without generation at all.

2.6 EgoLife and EgoLifeQA as the evaluation setting

EgoLife [2] is a long-form egocentric dataset in which six participants were recorded continuously with wearable cameras while living together for a week, producing approximately 300 hours of synchronised first-person video with dense captions and transcripts. EgoLifeQA is its accompanying benchmark: long-context question answering over each participant's own recorded life, with questions that require locating and combining evidence across hours or days.

Three properties of the dataset matter operationally here. It is distributed as pre-sliced video files of approximately 30 seconds, which as established is the source of the baseline granularity. Its caption and transcript layer is dense enough that a text-only memory is informative, which is what makes the retrieval evaluation in Section 4.8 possible without visual retrieval. And it is genuinely long and genuinely repetitive, since much of a week of ordinary life is spent in a small number of rooms doing a small number of things: which is what makes it a fair test of a consolidation method rather than a favourable one.

Ego4D [28] is the broader context: 3,670 hours of egocentric video across hundreds of participants, with an episodic-memory benchmark suite. It is larger and more diverse but not continuous

per participant in the way EgoLife is, so it is less suited to a question about consolidating a single continuous life-log.

2.7 How systems of this kind are evaluated

The evaluation design in Chapter 4 follows from a gap in how this literature reports itself, so it is worth stating what the convention is.

Long-video memory systems are almost always evaluated on question-answering accuracy over a benchmark, and improvements are reported as accuracy gains: WorldMM, for instance, reports an average gain of 8.4% over prior state of the art across five long-video question-answering benchmarks [1]. Ablations within such papers typically remove a memory bank or a retrieval stage and report the resulting drop. What is rarely reported is what the memory costs to build, to store or to load, even though those quantities determine whether a system can run over days of video rather than minutes.

That convention is a poor fit for the change made here, for two reasons. First, accuracy on a QA benchmark measures the retriever and the generator as much as the memory; an indexing change can be invisible behind a strong reader, or be swamped by a weak one. Second, the purpose of event-boundary indexing is not to answer more questions but to represent the same experience in fewer, better-formed units, which is an efficiency claim that accuracy cannot express. Objective 3 is therefore framed around compression, database size and latency, with accuracy used as a **guard** rather than as the headline: the question asked of it is whether the cheaper memory answers as well, not whether it answers better.

This in turn dictates the statistical treatment. Because both arms answer the same questions under the same retrieval policy, the outcomes are paired rather than independent, and the appropriate test on paired binary outcomes is McNemar's [30], applied in its exact form because the discordant counts are small. It also dictates how the result must be read: a test of this kind can fail to find a difference either because there is none or because it lacks the power to see one, and Section 4.8.1 quantifies which applies here rather than allowing the null result to stand as evidence of equivalence.

2.8 Summary and the gap addressed

The literature supports four statements. Long-form video memory systems exist and are converging on a retrieve-then-generate architecture with structured banks [1], [11], [12], [13]. Their segmentation is uniform and, in the specific case of the strongest such system on egocentric video, is inherited from the dataset's file layout rather than chosen. Cheap, well-validated methods for content-driven temporal segmentation have been available for two decades [19], [20], [23] but have not been applied to the memory construction step of these systems. And the biological account of memory that these systems borrow their vocabulary from [4], [5], [7], [26] includes an offline

consolidation stage that the systems themselves do not implement: what WorldMM calls consolidation is, as Section 4.1 measures, accumulation. Table 2-1 places this work against the systems reviewed above.

System	Unit of memory	How the unit is chosen	Offline consolidation	Egocentric, multi-day
MovieChat [11]	fixed frame window	uniform	token-memory compression, online	no
MA-LMM [12]	fixed frame window	uniform	memory-bank compression, online	no
VideoAgent [13]	fixed segment	uniform	none	no
MemGPT [14]	observation / message	arrival	paging between tiers, no merging	not video
Generative agents [15]	single observation	arrival	reflection, one LLM call per synthesis	not video
HippoRAG [7]	text passage	corpus-given	none	not video
WorldMM [1]	one video file (~30 s)	dataset file layout	none — the semantic store accumulates (Section 4.1)	yes
This work	detected event (10–300 s)	visual change detection	offline, cycle-based, zero LLM calls	yes

Table 2-1 — Where the contribution sits. The two right-hand columns are the gap.

The gap is therefore narrow and concrete. **No existing long-form video memory system segments its stream on detected content change, and none performs a genuine offline consolidation of its semantic store; the strongest published system does neither, and the cost of doing neither has not been measured.** This dissertation implements both against that system's own pipeline and on its own data, and measures the cost of each in storage, in compute, and in retrieval accuracy: including where the measurement is unfavourable.

3 EVENT-BOUNDARY INDEXING

This chapter presents the first contribution. Section 3.1 describes the platform, the data, and the division of labour, so that later results can be read against a known setup. Section 3.2 documents the work required before any comparison was valid: an inherited defect that suppressed part of the baseline, and a data-integrity problem that forced the exclusion of one day. These are not incidental: the first changed the baseline the whole dissertation compares against. Sections 3.3 to 3.5 give the method, its configuration, and the sensitivity analysis from which the operating point was chosen. Section 3.6 reports the results that answer Objective 1.

3.1 The baseline system and the experimental platform

3.1.1 How the baseline builds its memory

Everything in this chapter is a modification of an existing pipeline, so that pipeline is described first, in the order the data moves through it. Figure 3-1 is the reference.

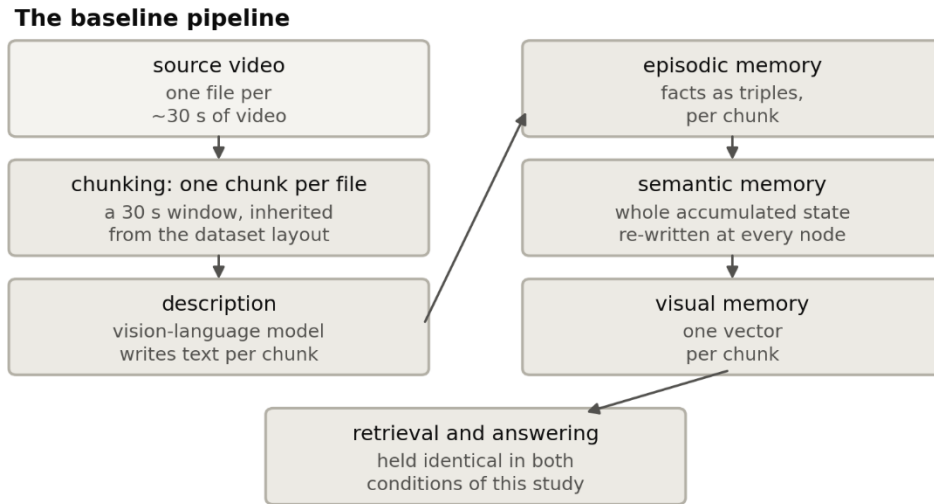


Figure 3-1 — *The baseline memory pipeline, as inherited. The retrieval stage at the bottom is held identical throughout this study, so it is shown once and not discussed further until Section 4.8*

Source video. The dataset arrives as a sequence of short files, each holding approximately 30 seconds of continuous recording, accompanied by dense captions and a transcript of speech.

Chunking. The pipeline forms one memory unit per file. It does not choose a window length; it consumes the layout it is given. Section 1.1.4 sets out why this matters more than it first appears.

Description. For each unit, a vision-language model is given the unit's caption and transcript text and asked to return structured output. This is the expensive stage, and the one that dominates the cost of building the memory.

Episodic memory. The model's output is a set of subject-relation-object facts for that unit, together with the entities mentioned. These are assembled into a graph and retrieved with a method derived from HippoRAG [7].

Semantic memory. Every ten consecutive units, the pipeline emits a semantic node intended to hold generalised knowledge accumulated so far. What it actually writes is the entire accumulated state, re-serialised in full at every node, which Section 4.1 measures.

Visual memory. Independently, each unit is encoded into a single 1536-dimensional embedding vector supporting visual similarity search.

Retrieval. At query time an agent searches the banks, retrieves a small number of entries, and answers from those alone.

3.1.2 What this work changes, and what it holds fixed

Two of those stages are replaced; the rest are left exactly as they are. Figure 3-2 shows the same pipeline with the changes marked.

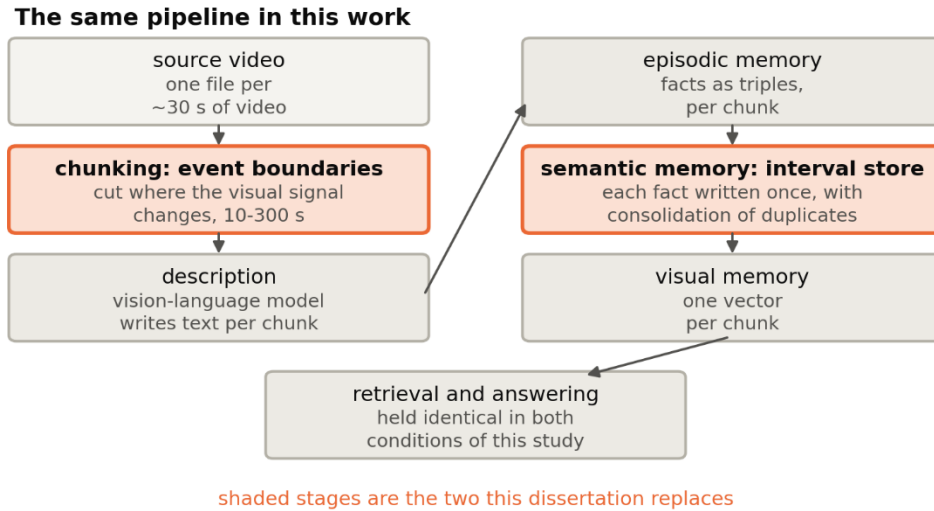


Figure 3-2 — *The same pipeline with this dissertation's two contributions in place. Only the two shaded stages differ from Figure 3-1.*

Chunking becomes boundary detection. Instead of one unit per file, the stream is stitched into a continuous per-day timeline and cut where the visual signal changes, subject to a minimum and maximum length. Chapter 3 from Section 3.3 onwards is about this stage.

The semantic store becomes an interval store, with consolidation. Instead of re-serialising the accumulated state at each node, each distinct fact is written once with the period over which it was observed, after an offline pass has merged near-duplicates. Chapter 4 is about this stage.

Everything else is identical between the two arms: the same source video, the same vision-

language model and prompts, the same embedding models, the same episodic extraction, the same visual encoding, and the same retrieval agent. That is what makes the comparison in this dissertation an ablation of the two changed stages rather than a comparison of two systems, and Section 3.1.3 sets out the controls that keep it so.

The experiment therefore has two arms:

- **FIXED:** the baseline. Chunks are 30-second windows, reproducing WorldMM's effective granularity.
- **EVENT:** the treatment. Chunks are delimited by detected event boundaries.

3.1.3 Experimental design and the controls applied

The design is a two-arm controlled ablation over a single manipulated variable. Its value depends entirely on what is held fixed, so Table 3-1 sets those choices out rather than leaving them implicit, and each is revisited where the corresponding result is reported.

Held constant across both arms	Why it matters
Source video, captions and transcripts	Both memories describe the same underlying experience
Vision-language model and its prompts	Extraction quality cannot differ between arms
Text and visual embedding models	Database sizes and similarity behaviour stay comparable
Extraction, semantic and consolidation code	Only the chunk boundaries differ downstream
Semantic node period (300 s of elapsed time)	Node size cannot confound the consolidation result (Section 4.4.5)
Retrieval policy and evaluation harness	Accuracy differences are attributable to memory, not to querying (Section 4.8)
Consolidation threshold (0.86)	Compression is compared at one operating point, not at two tuned ones

Table 3-1 — What is held constant across the two arms, and why each matters.

Four specific confounds were identified during the work and controlled rather than tolerated. Table 3-2 lists them together, because a reader is entitled to ask what would have gone wrong had they not been.

Confound	How it was controlled	Reported in
The inherited baseline was degraded by a	Both arms rebuilt from the raw	Section 3.2

Confound	How it was controlled	Reported in
silent extraction defect, so reusing it would have measured that defect as an effect of chunking	stream after the defect was fixed, at roughly 41 GPU-hours	
Building semantic nodes from a fixed number of chunks would give the event arm nodes about 2.7 times longer, and longer nodes compress better for reasons unrelated to the algorithm	Nodes built from elapsed time instead of chunk count	Section 4.4.5
Choosing the detection threshold by what maximised downstream accuracy would invalidate the evaluation that follows	Threshold fixed on the share of boundaries coming from detection, measured before any retrieval was run	Section 3.5.2
Attributing the segmentation to detection when a length cap could have produced it	Every boundary tagged with the rule that generated it, making the central claim falsifiable	Section 3.6.2

Table 3-2 — Confounds identified during the work, and how each was controlled..

Two differences could not be removed and are carried as limitations instead: the arms cover slightly different spans of the timeline for a structural reason (Section 3.1.5), and the event arm has a higher rate of chunks from which no triples could be extracted (Section 4.9, item 8). Both are reported wherever they bear on a result.

3.1.4 Attribution

Section 1.2.3 states the delimitation. In terms of the components a reader of this chapter will meet:

Built for this dissertation. The boundary detector and the frame-feature front end that supplies it; the chunk assembly stage and its content-addressed identifier scheme; the hardened extraction path of Section 3.2; the time-based semantic node builder; the interval-encoded semantic store and the consolidation algorithm of Chapter 4; and the metrics harness that produced every figure and table in this dissertation. Both experimental conditions, and all chunk, memory, consolidation, metric and evaluation outputs reported here, were produced by that work.

Not written by the author. The retrieval agent and evaluation harness used in Section 4.8, and a spatial memory bank that no experiment reported here exercises. Both were used exactly as received.

Upstream. The EgoLife video, captions and transcripts, and the reference system's own source release.

3.1.5 Corpus, hardware and models

Corpus. All experiments use a single EgoLife participant, A1_JAKE, days of continuous recording amounting to approximately **43 hours**. The source is 5,156 video files of roughly 30 seconds each, together with the dense captions and speech transcripts that accompany them, drawn from 97,867 caption and transcript entries. The accompanying EgoLifeQA benchmark provides 500 questions for this participant, of which the subset described in Section 4.8 is used.

The two conditions cover **40.02 h** and **44.13 h** of that recording respectively. The figures differ, and are not directly comparable, because fixed chunks are bounded by each source file's captions while event segments span the intervals between files. Chunk counts and, after canonicalisation, fact counts are therefore the comparable quantities, and aggregate duration is not. Section 3.4 returns to this.

Hardware and models. All compute ran on the University of Surrey Eureka cluster: two NVIDIA A100-80GB GPUs on a contended partition, MIG slices on a second node for smaller jobs, and a shared CPU partition for feature extraction. The models are held constant across conditions: Qwen3-VL-30B-A3B-Instruct for captioning, extraction and question answering; Qwen3-Embedding-4B for text embeddings; VLM2Vec-V2.0 [31] for visual embeddings. Model load time is 32–45 minutes depending on filesystem cache state; staging HF_HOME to parallel scratch reduced it from 45 to 32 minutes. Appendix B gives the full reproduction sequence.

3.2 Establishing a sound baseline

The finding in this section changed a result elsewhere in the dissertation, and is reported in the main text for that reason.

The inherited extraction stage turned the vision-language model's output into structured triples by parsing it as JSON. When the model produced output that would not parse, having been truncated at the token limit, wrapped in explanatory prose, or otherwise malformed, the parser caught the failure and returned a schema-valid **empty** result.

Three properties combined to make this invisible:

1. The retry mechanism fires on raised exceptions only. A successfully returned empty result is not an exception, so **no retry ever fired**.
2. Generation was deterministic, so even if a retry had fired it would have replayed byte-identical output.
3. The reference implementation contains a routine for repairing malformed model output that no code path calls. It was written and never connected, which is why truncated output was discarded rather than salvaged.

The failure mode is therefore not a crash but a quiet, systematic loss of content that reports

success. Measured on the inherited artifact: **680 of 5,156 build-set chunks (13.2%) returned zero triples, every one of them with non-empty caption text.**

The fix reverses each property. Truncated, malformed and prose-wrapped JSON is salvaged; parse failure is *reported* so that retries fire; generation settings escalate between attempts; and the empty result is produced only after the escalation plan is exhausted, where it is logged and counted rather than passing silently. A schema mismatch was also handled: the model occasionally nests a list where a string belongs, producing a triple whose object is itself a triple. Flattening these accounted for a further 86 recoveries.

Over 14,496 extraction calls in the rebuild, the recovery behaved as set out in Table 3-3:

	Count
Unparseable at attempt 1	473
Recovered by attempt 2 (sampling enabled)	232 (49%)
Recovered by attempt 3 (token limit doubled)	1 (0.4%)
Gave up after the full plan	240
Recovered by salvage without any retry	395

Table 3-3 — Recovery over 14,496 extraction calls in the rebuild.

Tier 3 is worth reporting as a negative engineering result: doubling the token limit recovered exactly one call out of 241 attempts, so truncation was almost never the actual cause, and the tier was replaced with a hotter sampling configuration.

After the fix the empty-chunk rate in the fixed condition fell from 13.2% to 3.63%, and the rebuilt baseline extracts 75,399 triples against the inherited 72,141 on identical chunks. The baseline used for comparison is therefore a somewhat stronger one than was inherited, which makes the Objective 1 and 3 comparisons harder rather than easier.

3.3 Boundary detection method

3.3.1 Design constraints

Three constraints shaped the method before any algorithm was chosen.

No vision-language model in the detection loop. The purpose of event-boundary indexing is to make the memory cheaper and better-formed. A detector that itself required a VLM forward pass per frame would cost more than the extraction it feeds, and the contribution would be self-defeating. Detection must be cheap enough to be free relative to the rest of the pipeline.

Detection must cross file boundaries. Because EgoLife's ~30 s files are an artifact of packaging rather than of content, a detector that ran within each file could never produce a chunk longer than

30 s and would reproduce the baseline. Frames are therefore stitched into one continuous per-day timeline, and the detector never sees a file boundary.

The output must be a partition, not a set of proposals. The downstream memory pipeline requires every second of the timeline to belong to exactly one chunk, so detection must be followed by an assembly stage with explicit rules for minimum length, maximum length and unobserved intervals.

3.3.2 Features

Each sampled frame is reduced to two cheap, complementary descriptors:

- an **HSV colour histogram** with $8 \times 8 \times 8$ bins, normalised, capturing the colour composition of the scene; and
- a **grayscale thumbnail** downsampled to 32×32 , capturing coarse spatial layout.

The colour histogram is invariant to spatial rearrangement and therefore robust to the constant small head motion of egocentric footage, but blind to a change in layout under constant lighting. The thumbnail is the reverse. Combining them covers both failure modes at negligible cost. Frames are sampled at **2 fps**, which bounds boundary localisation error at 0.5 s: an order of magnitude finer than the ± 15 s quantisation that snapping to the 30 s grid would have imposed.

3.3.3 Dissimilarity and adaptive thresholding

For adjacent sampled frames $i-1$ and i , let $\mathbf{h}_i$ be the normalised HSV histogram and $\mathbf{g}_i$ the flattened grayscale thumbnail. A scalar dissimilarity d_i is formed as a weighted combination of the two descriptor distances, with $w = \text{hist_weight} = 0.5$ giving equal weight to colour and layout:

$$d_i = w \|\mathbf{h}_i - \mathbf{h}_{i-1}\|_1 + (1 - w) \|\mathbf{g}_i - \mathbf{g}_{i-1}\|_1 \quad (3.1)$$

The raw signal cannot be thresholded absolutely. Egocentric dissimilarity has a strongly varying baseline: walking down a corridor produces continuously high d , while sitting at a desk produces continuously low d , and a boundary is a departure from the *local* baseline, not from a global constant. The threshold is therefore adaptive, computed over a rolling window W of 60 seconds using the median and the median absolute deviation (MAD) of the dissimilarities inside that window:

$$\text{MAD}_i = \text{median}_{j \in W} |d_j - \text{median}_{k \in W} d_k| \quad (3.2)$$

$$z_i = \frac{d_i - \text{median}_{k \in W} d_k}{1.4826 \cdot \text{MAD}_i} \quad (3.3)$$

MAD is used rather than standard deviation because the statistic must be estimated from a window that may itself contain boundaries; the standard deviation is inflated by the very outliers being detected, which suppresses exactly the events of interest, whereas MAD has a 50% breakdown point [29]. The constant 1.4826 makes the estimate consistent with the standard deviation for Gaussian

data, so the threshold can be read as a familiar z-score.

A frame is a boundary candidate when $\mathbf{z}_i > \tau$ and $\mathbf{d}_i > \delta$, with the threshold $\tau = 4.0$ selected in Section 3.5.2 and the absolute floor $\delta = 0.02$. The second condition suppresses a specific failure: during a very still period the local MAD collapses towards zero, and trivial sensor noise acquires a large z-score. The absolute floor prevents the detector from firing on nothing.

3.3.4 Why not a learned boundary model

A GEBD-style learned detector [23] would likely localise boundaries more accurately. It was not used, for three reasons. It reintroduces a per-frame neural forward pass, violating the first design constraint. It introduces a domain-transfer question, GEBD models are trained on curated third-person video, not on continuous head-mounted footage — that would need its own validation and would confound the ablation. And the evaluation in this dissertation is of the *downstream consequences* of segmentation, not of boundary localisation accuracy against human annotation; there are no human boundary annotations for EgoLife, so a more accurate detector could not be shown to be more accurate here. Section 5.2 returns to this as the clearest line of future work.

3.3.5 Computational cost

The detector's cost is linear in the number of sampled frames and constant in memory. Over T seconds sampled at r frames per second there are rT frames; each contributes one histogram of $B = 8 \times 8 \times 8 = 512$ bins and one thumbnail of $32 \times 32 = 1,024$ values, so feature extraction is $\Theta(rT)$ in time and $\Theta(B + 1,024)$ per frame in space. Differencing is $\Theta(1)$ per adjacent pair. The rolling median and MAD over a window of $rW = 120$ samples are $\Theta(rW \log rW)$ per frame in the naive implementation used here, which at $rW = 120$ is a small constant.

For the full corpus this is 43 hours $\times$ 2 fps $\approx$ 310,000 sampled frames, and the measured end-to-end cost was **36.5 minutes of wall-clock on 16 CPU cores, with zero failures** (Section 4.7). Against the 17.9 GPU-hours the event condition then spends on extraction, the detection front-end is approximately 0.6% of pipeline cost on hardware that is not the bottleneck resource. The first design constraint of Section 3.3.1, that detection must be free relative to what it feeds, is therefore met with a wide margin, and the margin is what makes the choice of a classical rather than a learned detector defensible rather than merely convenient.

3.4 From boundaries to chunks

Detected boundaries are converted into a partition by four rules, applied in order.

1. **Minimum length** (min_event_seconds = 10). Boundaries closer than 10 s to the previous one are suppressed. Without this, a burst of rapid head movement fragments into unusably short chunks.

2. **Maximum length** (`max_event_seconds = 300`). If no boundary has been detected for 300 s, a cut is forced. This bounds the worst case for a stationary camera pointed at an unchanging scene, and bounds the retrieval-context size any single chunk can produce.
3. **Gap handling** (`gap_tolerance_seconds = 2.0`). Where the timeline has no video, the camera was off, or a file is missing: a cut is forced at the discontinuity, so that no chunk silently spans unobserved time.
4. **Text aggregation**. Each resulting segment's text is the concatenation of every dense caption and transcript line whose timestamp falls inside it.

Every cut is tagged with the rule that produced it, as detected, cap or gap. This tagging is what makes the central Objective 1 claim measurable rather than assumed, without it, a result of "2,092 chunks" would be consistent with a detector that never fired and a 300 s cap doing all the work.

Segments that contain video but no text at all are dropped, since they cannot contribute to a text-based memory. **309 segments (about 13%) were dropped on this basis**: spans where both dense captions and transcripts are absent. This accounts for the difference between 2,401 segments and 2,092 chunks.

It is worth being explicit that this does not discard content the baseline retains. Fixed chunks are constructed from the caption stream and so exist only where captions exist; a span with no captions and no transcript produces no fixed chunk either. The 309 dropped segments are intervals that are absent from *both* memories, and the asymmetry is one of bookkeeping: the event pipeline counts them and then removes them, whereas the fixed pipeline never enumerates them at all. This is also why the event condition's nominal coverage (44.13 h) exceeds the fixed condition's (40.02 h) while neither memory contains more of the underlying stream than the captions support.

3.5 Configuration and sensitivity analysis

3.5.1 Final configuration

Parameter	Value
threshold (z-score)	4.0
min_event_seconds	10
max_event_seconds	300
sample_fps	2.0
hist_bins	(8, 8, 8)
gray_size	32
hist_weight	0.5

Parameter	Value
min_absolute_distance	0.02
rolling_window_seconds	60
gap_tolerance_seconds	2.0

Table 3-4 — Detection configuration used for the event condition.

3.5.2 Threshold sensitivity

The z-score threshold is the only parameter that materially changes the output, so it was swept across seven values in two regimes: with the maximum-length cap raised to 600 s, to isolate the detector's own behaviour, and at the final 300 s cap. Figure 3-3 plots both.

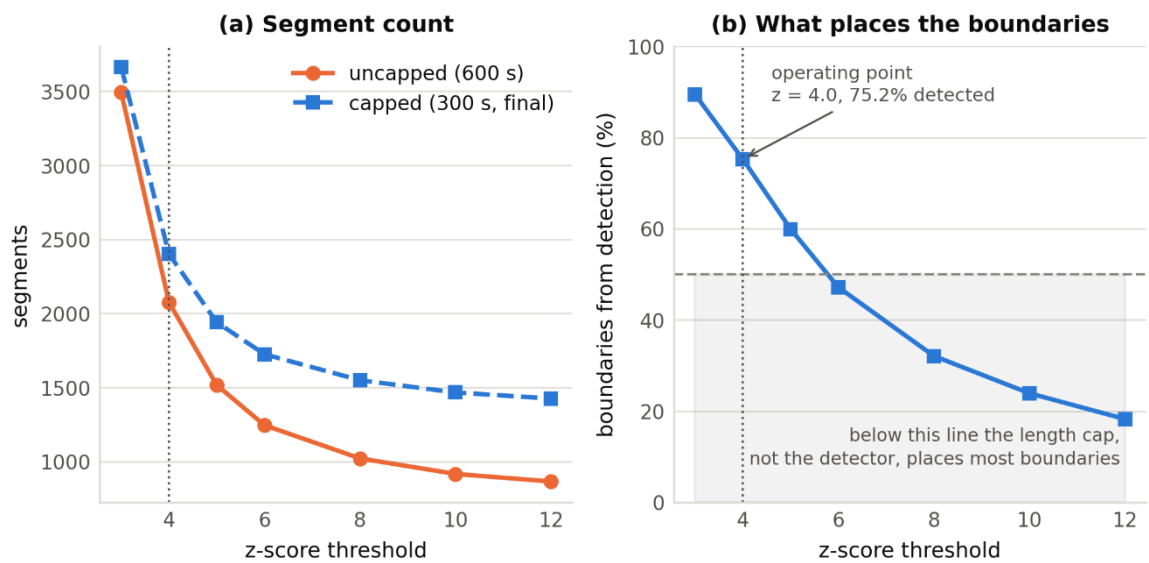


Figure 3-3 — Sensitivity of the segmentation to the z-score threshold. (a) Segment count in both cap regimes; the curve decays smoothly and does not by itself single out an operating point. (b) The share of boundaries actually placed by detection, which does: it crosses 50% between $z = 5$ and $z = 6$, beyond which the length cap rather than the detector is determining the segmentation.

z-thresh-old	uncapped seg-ments	uncapped mean (s)	capped seg-ments	capped mean (s)
3.0	3,492	69.2	3,663	65.9
4.0	2,075	116.4	2,401	100.6
5.0	1,516	159.3	1,940	124.5
6.0	1,245	194.0	1,724	140.1
8.0	1,021	236.6	1,549	156.0
10.0	916	263.7	1,466	164.8

z-thresh- old	uncapped seg- ments	uncapped mean (s)	capped seg- ments	capped mean (s)
12.0	865	279.3	1,425	169.5

Table 3-5 — Threshold sweep. Uncapped isolates the detector; capped is the final configuration.

The threshold was selected on the last column, not on the shape of the segment-count curve. It is tempting to describe $z = 4$ as the knee of the count curve, and an earlier draft of this work did so, but that description does not survive scrutiny. The sweep begins at $z = 3$, so the curve's steep left-hand region was never observed; and a standard maximum-curvature test on the uncapped points places the elbow at $z = 5$ rather than $z = 4$. The count curve decays smoothly across the whole swept range and does not identify a single defensible operating point on its own.

What does identify one is the question of *which mechanism is placing the boundaries*. Objective 1 asks for a segmentation driven by change in the content, so a configuration in which the maximum-length cap places most of the cuts fails the objective however few segments it produces. That share is measured, and it falls steeply as the threshold rises: 89.4% of boundaries come from detection at $z = 3$, 75.2% at $z = 4$ and 59.9% at $z = 5$, and by $z = 6$ the cap is placing more boundaries than the detector is. **Threshold 4.0 is the largest value at which detection still accounts for three quarters of the segmentation**, and it was chosen on that basis.

Two properties of the choice are worth stating. It sacrifices compression deliberately: $z = 5$ would give 1,940 segments against 2,401, a further 19% reduction, at the cost of a quarter of the boundaries becoming arbitrary. And it is made on a quantity independent of the evaluation in Chapter 4, so the segmentation is not tuned against the metric later used to judge it, which would have compromised that evaluation.

3.5.3 Configurations rejected during development

Two earlier configurations were run and discarded before the final one; Table 3-6 records them, because the first is instructive about how the result could have been reached for the wrong reason.

Configuration	Chunks	Mean (s)	Why rejected
threshold 3.0, cap 60 s	4,922	31.4	Cap was binding; result nearly indistinguishable from the baseline
threshold 3.0, cap 600 s	3,327	49.2	Diagnostic only; used to establish the true length distribution
threshold 4.0, cap 300 s	2,092	75.9	final

Table 3-6 — Configurations evaluated during method development.

The first of these is instructive and is reported because it nearly produced a null result for the wrong reason. A 43-hour timeline under a 60 s cap cannot yield fewer than about 2,580 segments no matter what the detector does, so at that cap the *cap* rather than the detection was determining the outcome, and the resulting chunks were nearly indistinguishable from the 30 s grid. Recognising that the constraint rather than the method was binding is what motivated the diagnostic uncapped run and, from it, the 300 s operating cap.

3.6 Indexing results

3.6.1 Chunk statistics

	FIXED (baseline)	EVENT	change
chunks	5,156	2,092	−59.4%
mean length	27.9 s	75.9 s	2.7x
median length	29.0 s	37.5 s	
p05 / p95 length	24 / 30 s	10.5 / 260 s	
min / max length	1 / 30 s	10 / 300 s	
mean text	566 chars	1,373 chars	2.4x
median text	551 chars	591 chars	
p95 / p99 text	954 / —	5,026 / 7,055	
max text	12,716	13,234	
chunks spanning more than one source file	0	1,582 (75.6%)	
duplicate chunk ids	0	1	benign; see below

Table 3-7 — Chunk statistics, both conditions.

Three observations.

The distribution is heavily right-skewed and the mean alone is misleading. Median event length is 37.5 s against a mean of 75.9 s, with p95 at 260 s. The median event chunk is only about 30% longer than a fixed chunk; the mean is inflated by a minority of long, low-change stretches, sleep, sitting, screen work, which is exactly the repetitive material that Objective 1 predicts should be aggregated. The same asymmetry appears in the text lengths: median text rises only from 551 to 591 characters, while the mean rises from 566 to 1,373. Figure 3-4 shows both distributions.

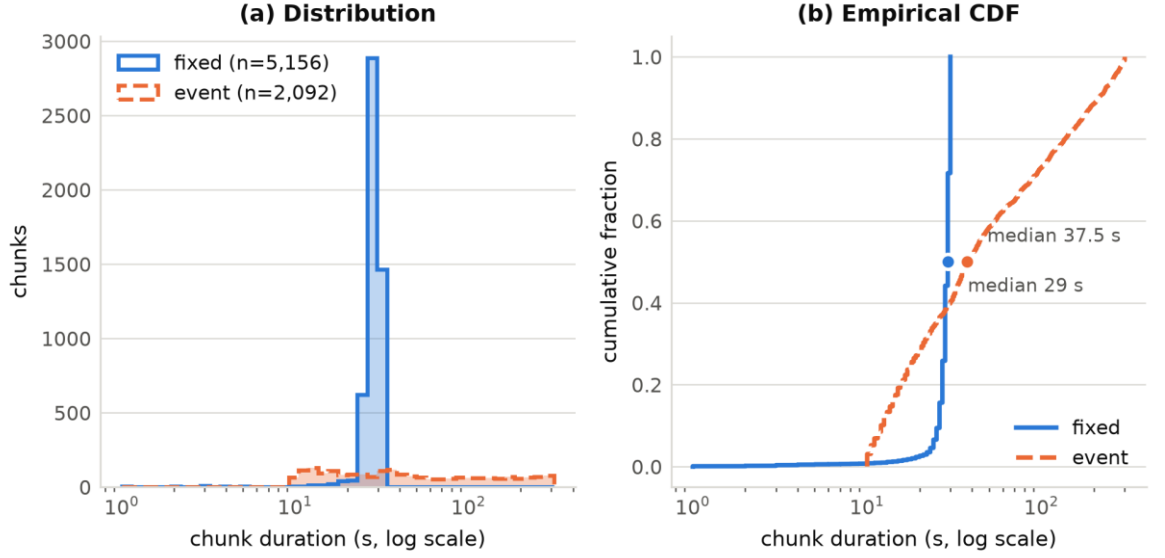


Figure 3-4 — *Chunk-length distribution (a) and empirical CDF (b), both conditions, log axis. The fixed condition is a near-degenerate distribution at 30 s. The event condition retains a mode near 30 s but adds a long right tail extending to the 300 s cap.*

1,582 of 2,092 event chunks (75.6%) span more than one source video file, against zero in the baseline. This is the direct, measurable consequence of stitching the timeline before detection, and it is the mechanism by which the method escapes the dataset-imposed granularity.

One duplicate chunk id occurred in the event condition. Two chunks aggregated to byte-identical text, so they share an md5 content hash and the extraction step returns a single result that both timestamps map to. This is correct content-addressed behaviour rather than a defect, and it affects one chunk in 2,092.

3.6.2 Boundary provenance

The central claim of Objective 1 is that the segmentation is driven by detected content change. Because every cut carries its origin, this is a measurement:

origin	count	share
detected (visual z-score outlier)	1,802	75.2%
cap (300 s maximum)	553	23.1%
gap (unobserved video)	40	1.7%

Per day:

Day	Source videos	Segments	Mean (s)	Detected	Cap	Gap
DAY1	808	416	94.6	306	92	17
DAY2	852	397	110.9	276	115	5

Day	Source videos	Segments	Mean (s)	Detected	Cap	Gap
DAY3	908	388	107.3	290	93	4
DAY4	949	460	90.8	375	81	3
DAY5	938	402	111.3	281	114	6
DAY7	701	338	88.7	274	58	5
total	5,156	2,401		1,802	553	40

Table 3-8 — Boundary provenance by day.

Figure 3-5 plots the same breakdown.

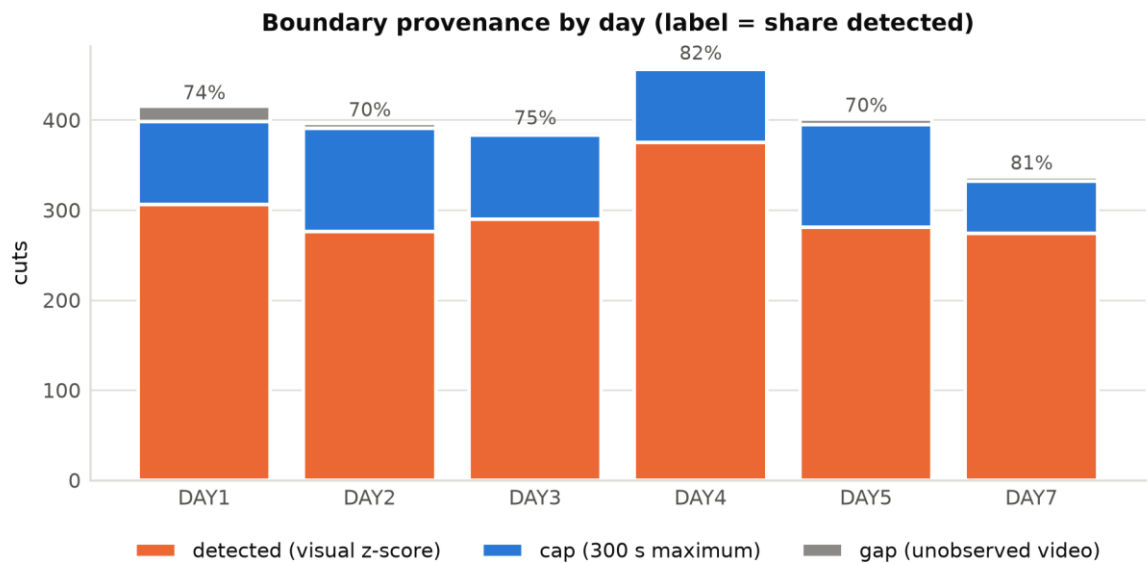


Figure 3-5 — *Boundary provenance by day; the label above each bar is that day's share of cuts produced by detection.*

Three quarters of boundaries come from detection, and the proportion is stable across days (70% to 82%), so the result is not carried by one atypical day. The 23.1% produced by the cap is the honest cost of bounding worst-case chunk length: those are stretches in which the detector found no change for five minutes, which on inspection are periods of sleep or sustained stationary activity. The 40 gap cuts confirm that unobserved intervals are handled explicitly rather than being silently absorbed into a neighbouring chunk.

Note also that the source-video column sums to 5,156: exactly the fixed condition's chunk count. This is the arithmetic confirmation of the correction made in Section 1.1: the baseline granularity is one chunk per shipped file.

Figure 3-6 shows what the two segmentations look like side by side over one ten-minute window of recording.

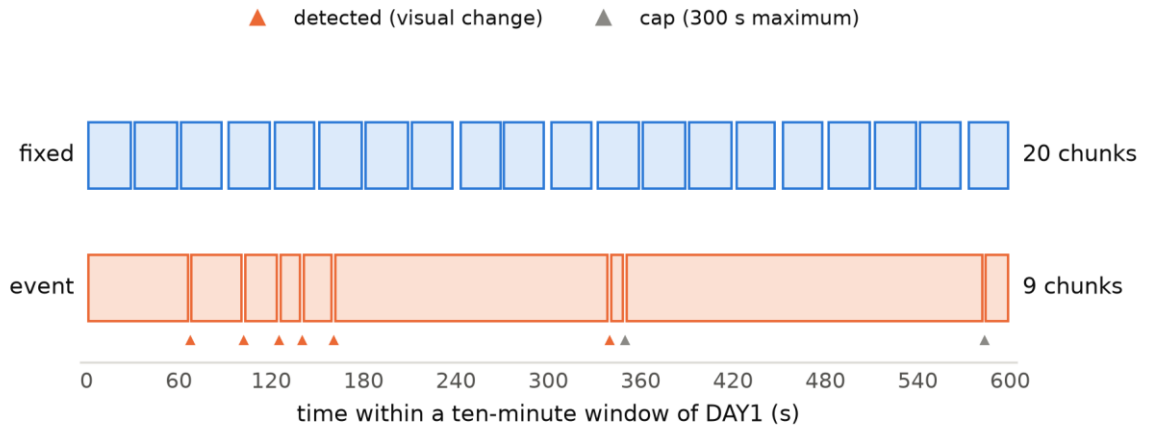


Figure 3-6 — *An arbitrary ten-minute window of DAY1 under both segmentations. The fixed grid cuts twenty times at regular intervals regardless of content. The event condition cuts eight times where activity is changing.*

3.6.3 How far the visual proxy can be trusted

Objective 1 speaks of "semantic state changes"; the detector responds to *visual* change. The two are not the same thing, and no human boundary annotations exist for EgoLife against which the proxy could be scored directly. That limitation is stated plainly in Section 5.1. What can be offered is indirect evidence, and it is worth setting out because an unvalidated proxy is the most obvious point of attack on this chapter.

The detector responds to structure, not to noise. A detector firing on sensor noise would show no stable relationship between threshold and output. The sweep in Table 3-5 is smooth and monotone across seven thresholds spanning a factor of four, in both cap regimes. A noise process would produce a curve governed only by the tail of the noise distribution, and in particular the uncapped mean segment length would not rise from 69.2 s to 279.3 s in the orderly fashion it does.

The result is stable across days. The detected share of cuts ranges from 70% to 82% across the six days (Table 3-8), which are different days with different activities. A detector dominated by an artefact of one recording session would not behave this consistently.

The boundaries fall where the constraints do not put them. 75.2% of cuts come from detection rather than from the cap or from gaps, and 75.6% of resulting chunks cross a source-file boundary. Neither number is achievable by any degenerate detector: firing never gives 0% detected, and firing constantly gives chunks pinned at the 10 s minimum rather than a median of 37.5 s.

The downstream consequence is neutral rather than harmful. Section 4.8 finds identical retrieval accuracy on the questions both configurations answered. A segmentation that cut through the middle of the evidence needed to answer questions would be expected to degrade accuracy, and it does not.

Taken together this establishes that the detector is measuring something real and stable in the

video, and that using its output as the unit of memory costs nothing measurable in retrieval. It does **not** establish that its boundaries coincide with the boundaries a human would draw, which is a different claim requiring different evidence. Section 5.2 sets out the annotation exercise that would supply it.

3.7 Summary

Objective 1 is met and the result is measured rather than asserted. Segmenting on visual change reduced the chunk count by 59.4% over 43 hours, with 75.2% of the boundaries produced by detection, at 2 fps on CPU with no vision-language model involved. Three quarters of the resulting chunks cross the dataset's file boundaries, which the baseline by construction cannot do. The operating threshold was chosen on a measured property of the segmentation itself, the share of boundaries coming from detection, rather than tuned against a downstream metric. An engineering defect that would have invalidated the comparison, a silent parser failure in the inherited extraction stage, was found by measurement and is documented here because correcting it materially strengthened the baseline being compared against.

Section 3.6.3 sets out how far the visual signal can be trusted as a proxy for semantic change. The evidence that the detector responds to real structure is strong, resting on an orderly threshold response, stability across six days, and boundaries the constraints could not have placed. But there are no human boundary annotations for EgoLife, so no claim is made that the detected boundaries coincide with those a person would draw.

What this chapter does *not* establish is whether the resulting memory is better. That is Chapter 4.

4 MEMORY CONSOLIDATION AND SYSTEM EVALUATION

This chapter presents the second contribution and the complete evaluation. Section 4.1 measures what WorldMM's semantic consolidation actually does, which converts the premise of Objective 2 from an assertion in the proposal into a measurement on this project's own data. Sections 4.2 and 4.3 present the interval-encoded store and the sleep-cycle algorithm. Sections 4.4 to 4.8 report the Objective 3 evaluation in full: compression, database size, latency, extraction cost and retrieval accuracy. Section 4.9 states the threats to validity. The results that do not favour the approach are in the main sequence, not in an appendix.

4.1 What WorldMM's consolidation actually does

The inherited artifact `semantic_consolidation_results_qwen3vl-30b.json` is a complete WorldMM semantic consolidation run over subject A1_JAKE, produced by an earlier build on the shared code-base. It is used here as a **behavioural reference**, not as a size baseline, and Table 4-1 records what it does: it was produced on a different, larger input than the rebuilt conditions, so its absolute size is not comparable to theirs. What it establishes is what the reference algorithm *does*.

semantic nodes	623
final state	1,582 triples
peak state	1,644 triples
triples written to disk	611,350
write amplification	372x
steps non-decreasing	$547 / 622 = 87.9\%$
file size	56.2 MB

Table 4-1 — WorldMM's measured consolidation behaviour on the inherited run.

Figure 4-1 plots the trajectory that produced those figures.

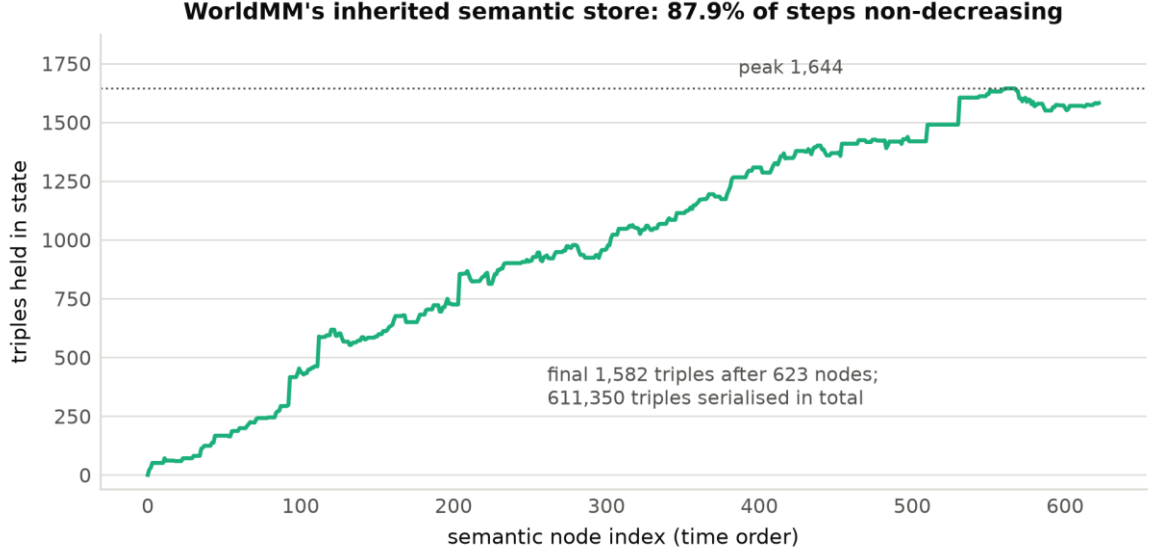


Figure 4-1 — Growth of WorldMM's inherited semantic store across its 623 nodes. The state rises monotonically to a peak of 1,644 triples; the small dips are local rewrites. 611,350 triples were serialised in total to record a final state of 1,582.

Two things follow. First, **the store grows monotonically**: 87.9% of steps are non-decreasing, and the trajectory is a steady climb with no sustained compaction. What the system calls consolidation does not reduce anything. Second, **each node re-writes the whole accumulated state**, so recording a final state of 1,582 triples required writing 611,350: a measured write amplification of 372x, and a file of 56.2 MB to hold about fifteen hundred facts.

An upper bound on write amplification can also be computed, by assuming the state is rewritten at every node and that nothing is ever merged. That model yields 272x for the fixed condition and 349x for the event condition. It is not used anywhere in this dissertation, because the reference implementation does merge, and the model consequently overstates its real output by a factor of 2.4: it predicts 1,478,203 written triples against an actual 611,350. The figures reported here are the measured 372x for the inherited run and the like-for-like storage comparison of Section 4.4.1.

4.1.1 Why the facts cannot merge

There is a second, more specific finding in the same artifact. Its 1,582 final triples carry **1,469 distinct predicates**. Nearly every fact has a unique free-text relation phrase. Because the store's only notion of identity is string equality over the triple, near-duplicate facts — the same observation phrased three slightly different ways across three nodes — can never merge. The unnormalised relation vocabulary described in Section 2.3 is not a theoretical concern here; it is the concrete, measured mechanism by which the store fails to compress, and it is the target the algorithm in Section 4.3 has to hit.

4.2 The interval-encoded semantic store

The storage problem in Section 4.1 is a problem of *encoding*, and it is separable from the problem of *compression*. Splitting them is what makes the result in Section 4.4.1 interpretable.

WorldMM's format answers the query "what did the system know at time t ?" by storing, for every node, the complete state as of that node. Reading is trivial and writing is quadratic.

The interval store inverts this. Each distinct fact is written **once**, annotated with *first_seen* and *last_seen* timestamps and a support count of how many observations it aggregates:

(subject, predicate, object, first_seen, last_seen, support)

Writing I for the set of stored facts, the state at any time t is not stored at all but recovered on demand, by keeping the facts that had been observed by then:

$$S(t) = \{ f \in I : \text{first_seen}(f) \leq t \} \quad (4.1)$$

Equation (4.1) is the whole of the encoding. It is also what Section 4.6.2 tests: if $S(t)$ returned anything different from the state the reference format stores explicitly at t , the two would not index to the same size or in the same time, and they do. Writing is linear in the number of distinct facts; reading requires one pass over the store.

4.2.1 Where the storage reduction comes from

A reduction of this size invites the suspicion that something has been left out, so it is worth setting out where it comes from before reporting it. The short answer is that it is a property of the file format rather than a compression result, and it can be predicted from two quantities that are measured separately.

Let N be the number of semantic nodes, s_n the number of triples held in the state at node n , and F the number of distinct facts in the final store. WorldMM writes the whole state at every node, so the total number of triple-records it serialises is

$$R_{\text{ref}} = \sum_{n=1}^N s_n \quad (4.2)$$

whereas the interval store writes each distinct fact exactly once:

$$R_{\text{interval}} = F \quad (4.3)$$

Because s_n is non-decreasing in n (Section 4.1), R_{ref} grows with the product of the node count and the mean state size, and therefore quadratically in N when the state grows roughly linearly with the node index. R_{interval} grows with F alone and is independent of N .

Records are not the same size, however. An interval record carries two timestamps, a support count and a variant list, so it is larger than a bare triple. Writing b_{ref} and b_{interval} for the mean bytes per record, the ratio of the two file sizes separates into two independent factors:

$$\text{reduction} = \frac{R_{\text{ref}}/R_{\text{interval}}}{b_{\text{interval}}/b_{\text{ref}}} \quad (4.4)$$

Measured on the artifacts, both terms are available separately, and Table 4-2 sets them out:

	FIXED	EVENT
Triple-records serialised, reference format	1,553,603	2,341,514
Rows in the interval store	5,366	6,398
Ratio of record counts	289.5x	366.0x
Bytes per record, reference format	93.4 B	91.8 B
Bytes per record, interval store	266.8 B	261.6 B
Size penalty per record	2.86x	2.85x
net reduction, equation (4.3)	101.4x	128.4x
net reduction, measured on disk	101.4x	128.4x

Table 4-2 — The storage reduction separated into its two causes.

This is the honest shape of the result, and it is a modest claim once stated properly. The interval store is **not** a more compact record format. Each of its rows costs 2.86 times as many bytes as a plain triple, because it carries the interval metadata that makes reconstruction possible. The whole of the gain comes from writing far fewer records, and that in turn is a consequence of the reference format's choice to re-serialise its entire state at every node.

Two limits follow directly. The reduction is a function of the node count, so it grows as the memory runs longer and would be small over a handful of nodes; it is not a fixed property of the method. And the event condition's larger figure, 128.4x against 101.4x, follows from its having 684 nodes against 534 rather than from event chunking aiding consolidation, which Section 4.4.2 shows it does not.

This is a classical time-interval encoding, and the trade it makes is explicit: writes and storage become linear, and reads acquire a filtering cost. Section 4.6 measures both sides of that trade, including where it does not pay off.

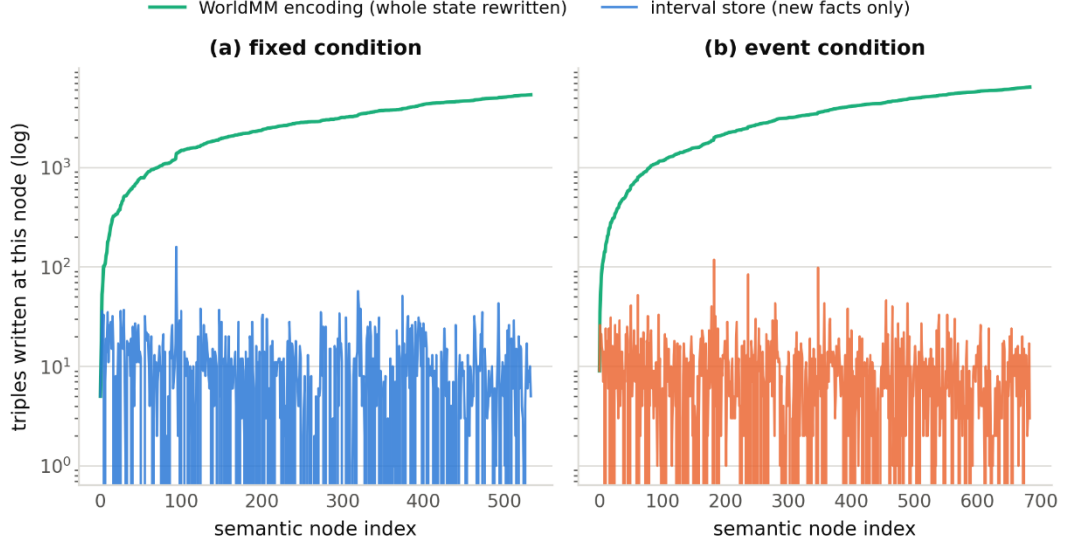


Figure 4-2 — *Triples written at each semantic node under the two encodings, log scale, for both conditions. WorldMM's encoding writes the whole state, which grows to 5,366 triples (fixed) and 6,398 (event). The interval store writes only facts not already present, which stays flat at a mean of 10.0 and 9.4 triples per node respectively across the whole run.*

Figure 4-2 is the mechanism behind every storage number in this chapter. The green curve is what the reference format writes; the flat trace is what the interval store writes. The gap between them widens monotonically with elapsed time, which is precisely the property that makes the reference format unsuitable for a memory intended to run indefinitely.

4.3 The sleep-cycle consolidation algorithm

Consolidation is an offline pass, run when the system is not serving queries, that takes the full set of extracted semantic triples and produces a smaller canonical set in which each retained triple carries the support of everything it absorbed. It runs in cycles, in the spirit of successive sleep cycles progressively integrating experience [4], [6], until no further merges occur.

The pass. For each cycle:

1. **Exact de-duplication.** Triples identical as strings are collapsed, their supports summed and their intervals unioned. On the fixed condition this alone reduces 8,757 extracted triples to 8,239: a small effect, which is itself the evidence for Section 4.1.1's point that string equality is not enough.
2. **Embedding.** Each remaining triple is embedded as a whole, subject, predicate and object together, not separately: with Qwen3-Embedding-4B. Embedding the whole triple is what allows two facts to merge when they differ only in how the relation is phrased, which is the specific failure mode identified in Section 4.1.1.
3. **Clustering.** Triples whose cosine similarity exceeds the threshold are grouped.
4. **Representative selection.** The representative of each cluster is **an existing member**, the one

with the highest support, breaking ties towards the shortest surface form. Nothing is generated.

5. **Aggregation.** The cluster's supports are summed and its intervals unioned onto the representative.

Cycles repeat until a fixed point; at the operating threshold this takes five cycles for the fixed condition and converges similarly for the event condition.

Algorithm 4-1 Sleep-cycle consolidation

```

Input : S      set of extracted semantic triples, each with a timestamp
         $\tau$     cosine similarity threshold (0.86)
        E      text embedding model
Output: I      interval store: distinct facts with first_seen, last_seen,
support
1  I <- {}
2  for each triple t in S ordered by timestamp:
3      if t already in I:                # exact string match
4          I[t].support <- I[t].support + 1
5          I[t].last_seen <- max(I[t].last_seen, time(t))
6      else:
7          I <- I union { (t, first_seen=time(t), last_seen=time(t),
support=1) }
8
9  repeat                                # one sleep cycle
10     V <- E(concat(subject, predicate, object)) for every row in I
11     C <- clusters of I where pairwise cosine(V) >  $\tau$ 
12     for each cluster c in C with |c| > 1:
13         r <- argmax_{x in c} ( support(x), -len(surface(x)) ) # an
existing member
14         support(r) <- sum of support over c
15         first_seen(r) <- min of first_seen over c
16         last_seen(r) <- max of last_seen over c
17         variants(r) <- variants(r) union (c \ {r})
18         I <- (I \ c) union {r}
19 until no cluster in C has |c| > 1
20 return I

```

Complexity. Lines 1–7 are $\Theta(|S|)$ with hashing. Each cycle embeds the surviving rows, $\Theta(|I|)$ model calls, and forms the pairwise similarity matrix, $\Theta(|I|^2)$ in time and space. Because $|I|$ shrinks monotonically across cycles, later cycles are cheaper, and the total is dominated by the first. This is precisely why the timings in Table 4-4 look inverted: a *low* threshold merges aggressively, so $|I|$ collapses quickly and few cycles are needed, whereas a high threshold leaves $|I|$ near its initial size and requires many cycles over a large matrix. At 0.94 the fixed condition ran 135 cycles against 5 at 0.86, which is the whole of the 76 s against 1.7 s difference. The quadratic term is also the reason the operating point is not chosen for speed: at this corpus size $|I| \approx 8,000$ and the matrix is trivial, but a store an order of magnitude larger would require blocking or approximate nearest-neighbour search rather than a dense matrix.

Line 13 is the design decision that removes generation from the loop, and line 17 is what makes

Section 4.4.4's audit possible: every absorbed triple is retained as a variant, so the merge is inspectable after the fact rather than being a lossy one-way operation.

The design decision that produces the cost result is step 4. WorldMM's semantic stage issues a language-model call per new triple to decide and phrase the merge. Selecting an existing cluster member instead means the algorithm needs no generative model at all — only an embedding model, which is already loaded for retrieval. This is the reason Section 4.4.3 reports zero language-model calls rather than a reduced number. It is also a genuine limitation, discussed in Section 4.9: an existing member is sometimes a worse phrasing than a generated summary would be, and paraphrases with no embedding overlap cannot be merged at all.

4.4 Consolidation results

4.4.1 Storage

The comparison below encodes **identical content** in both formats. Both hold the same consolidated triples for the same condition; only the serialisation differs. This isolates the encoding from the algorithm. Figure 4-3 shows the same comparison on a log scale.

	FIXED	EVENT
interval store	1.43 MB	1.67 MB
same content, WorldMM nested format	145.18 MB	214.88 MB
storage reduction	101.4x	128.4x

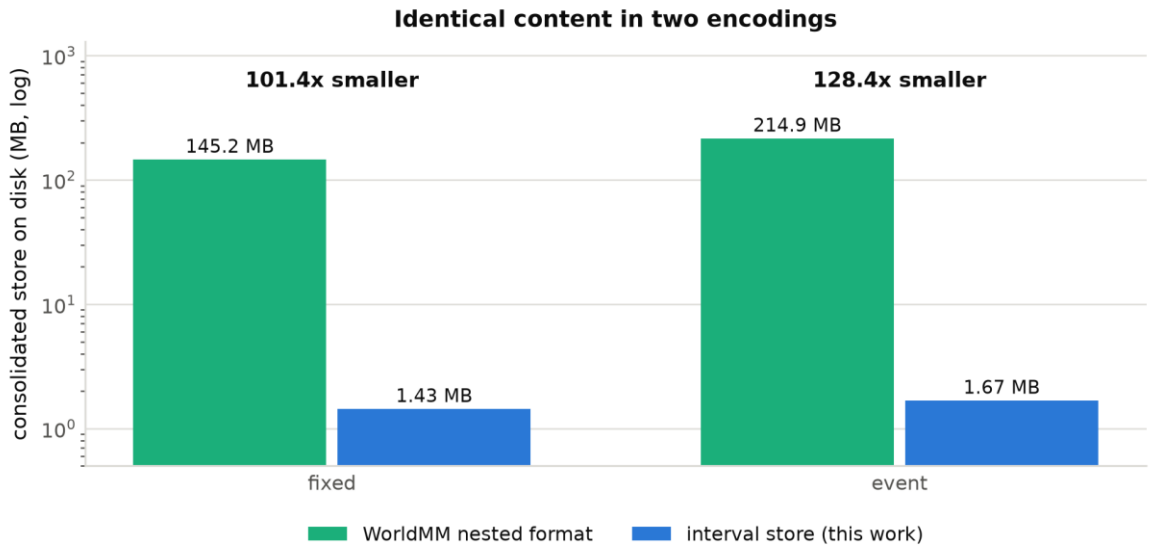


Figure 4-3 — *Consolidated semantic store on disk, identical content in two encodings, log scale.*

The reduction is larger in the event condition (128.4x against 101.4x) for a structural reason rather than a favourable one: the event condition has 684 semantic nodes against the fixed condition's 534, and the reference format's cost is quadratic in node count while the interval store's is linear in distinct

facts. More nodes therefore penalise the reference format more. This is worth stating plainly, because it means the difference between 101.4x and 128.4x is not evidence that event chunking helps consolidation: as Section 4.4.2 shows, it does not.

4.4.2 Compression, and the orthogonality result

Compression here is reported as **mean support**: the number of observations each retained fact stands for. For a consolidated store I in which every row carries the count of what it absorbed.

$$c = \frac{\sum_{f \in I} \text{support}(f)}{|I|} \quad (4.5)$$

This is worth stating explicitly because a natural alternative reading, extracted triples divided by retained triples, gives a different number. At the operating point the fixed condition retains 5,366 rows carrying 8,492 observations between them, so $c = 1.58$.

The similarity threshold was swept across five values in both conditions.

Thresh- old	FIXED retained	ratio	EVENT retained	ratio	max support (fixed / event)
0.78	2,961	2.87x	3,515	2.90x	482 / 634
0.82	4,136	2.05x	4,870	2.09x	205 / 285
0.86	5,366	1.58x	6,398	1.59x	103 / 103
0.90	6,606	1.29x	7,802	1.31x	34 / 69
0.94	7,576	1.12x	9,016	1.13x	18 / 34

Table 4-3 — Consolidation threshold sweep. The ratio is mean support: observations absorbed per retained triple.

Figure 4-4 plots the compression ratio and the largest merged cluster against the threshold.

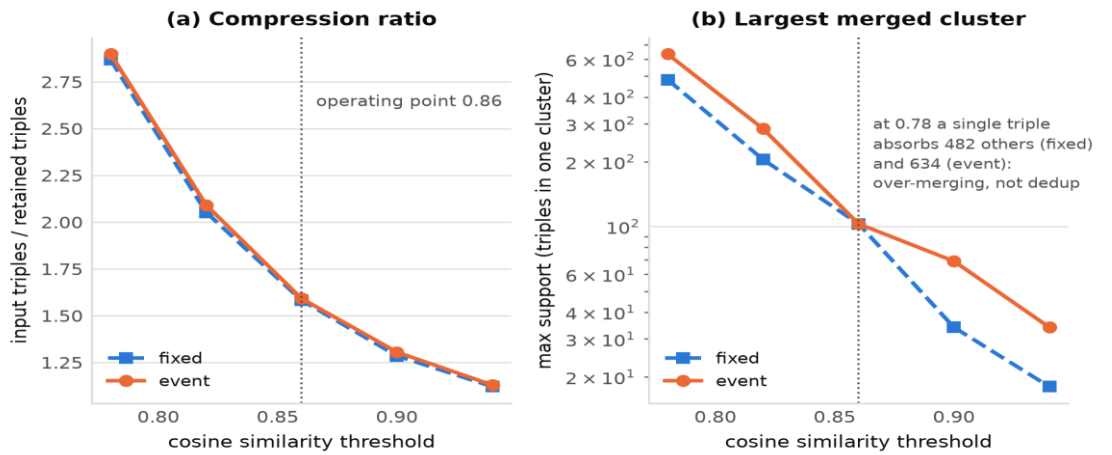


Figure 4-4 — Compression ratio (a) and largest merged cluster (b) against similarity threshold. The two conditions are indistinguishable in panel (a).

Threshold 0.86 was chosen as the operating point, governed by panel (b) rather than panel (a). Lower thresholds compress more, but the largest cluster grows rapidly: at 0.78 a single retained triple absorbs 482 others in the fixed condition and 634 in the event condition. A cluster of that size is not a de-duplicated fact, it is several distinct facts merged into one, and the compression it buys is destruction of content. At 0.86 the largest cluster is 103 in both conditions, and the retained store is 5,366 triples from 8,757 extracted.

The two conditions compress almost identically, 1.58x against 1.59x at the operating point, and 2.87x against 2.90x at the most aggressive setting. This is a negative result with respect to the project's original expectation. Consolidation was expected to compound with event-boundary chunking, on the reasoning that longer chunks would produce more internally redundant triple sets. It does not. **Consolidation is orthogonal to chunking strategy**: it operates on the semantic triple set, whose redundancy is a property of the underlying activity and of the extractor's phrasing habits, not of how the timeline was cut. The two contributions of this dissertation add; they do not multiply, and the dissertation does not claim that they do.

The orthogonality has one useful side effect, noted again in Section 4.9: it demonstrates that the compression metric is insensitive to node granularity, since the two conditions differ by 28% in node count and produce compression ratios differing by 0.6%.

4.4.3 Cost

	WorldMM	This work
language-model calls	one per new triple: ~8,757 (fixed), ~10,347 (event)	0
embedding calls	~245,000 (re-embeds all prior state per node)	8,249 (fixed), 9,795 (event)
wall-clock, full five- point sweep	—	513 s (fixed), 594 s (event)
wall-clock, single threshold	~9.7 h (estimated at 4 s/call)	1.2–399 s

Table 4-4 — Consolidation cost.

The elimination of generative calls follows directly from the representative-selection design in Section 4.3. At the measured extraction rate of about 4 s per call, WorldMM's per-triple merge scheme would take roughly 9.7 hours of serialised generation for the fixed condition alone; the consolidation stage here completes a five-point threshold sweep in 513 s.

The per-threshold timings are counter-intuitive and worth explaining, because they look like an

error otherwise. The first sweep point pays the full embedding cost (399 s); subsequent points reuse a cache, which is why 0.82 completes in 1.2 s. The highest thresholds are slower again, 76 s at 0.94 for fixed, 121 s for event, because fewer merges mean more surviving triples and therefore larger pairwise similarity matrices in each cycle.

4.4.4 Merge quality: what the compression actually costs

Compression ratios say nothing about whether the merges are *correct*. Because Algorithm 4-1 line 17 retains every absorbed triple as a variant, the store can be audited directly, and it should be: a consolidation method evaluated only on how much it removes is evaluated on the wrong axis.

At the operating threshold the fixed condition's store holds 5,366 rows, of which 928 are merged clusters absorbing 2,300 variants between them; the remaining 4,429 are singletons. The event condition holds 6,398 rows, 1,035 merged clusters and 2,519 variants.

Merges that work. The common case is the one the method was designed for: the same fact recorded with different relation phrasing, which string equality cannot catch:

```
("I", "follow instructions given by teammates", "")      support 4
<- ("I", "follow instructions given by team members", "")
<- ("I", "follow instructions given by others during collaborative activities", "")
<- ("I", "follow instructions given by team members effectively", "")
```

This is exactly the unnormalised-predicate problem of Section 4.1.1 being solved: four records collapse to one, no information is lost, and the interval preserves when the behaviour was observed.

Merges that do not. Inspection of the largest clusters shows two systematic failures.

```
("Lucia", "asks questions", "")      support 103
<- ("Lucia", "has uncertainty about friendship status", "")
<- ("Lucia", "considers suspects", "")
<- ("Lucia", "evaluates options critically", "")
("Myself", "moves through environment purposefully", "")  support 4
<- ("Alice", "moves within physical spaces purposefully", "")
<- ("Person I", "moves toward locations purposefully", "")
```

The first is over-generalisation: a cluster of 103 has absorbed distinct propositions that share only a broad topic. The second is worse, because it is a *category* error: the embedding is dominated by the predicate phrase, so two facts about **different people** merge. A store that has folded Alice's movement into the wearer's is not merely less precise, it is wrong.

This is measurable rather than anecdotal, and Table 4-5 quantifies it:

	FIXED	EVENT
rows in the consolidated store	5,366	6,398
rows that absorbed at least one variant	928	1,035

	FIXED	EVENT
variants absorbed	2,300	2,519
variants whose subject differs from their representative's	253 (11.0%)	361 (14.3%)
rows with an empty object field	3,984 (74.2%)	4,649 (72.7%)

Table 4-5 — Merge-quality audit of the consolidated stores at threshold 0.86.

About one merge in nine is across subjects (11.0% fixed, 14.3% event). Not all of these are errors, since ("*Alice*", "*assists*", "*Tasha*") legitimately absorbs ("*Tasha*", "*interacts_with*", "*Alice*"), which is the same relation viewed from the other side, but the movement example above shows that many are. Whole-triple embedding was chosen (Section 4.3, step 2) precisely so that predicate phrasing would not block a merge; the cost of that choice is that predicate phrasing can now *drive* a merge past a subject mismatch. **The obvious remedy is to constrain clustering to within a subject, or to weight the subject term in the embedding, and this was not implemented.** Section 5.2 lists it as the highest-value correction available to this method, and it is a limitation of the work as submitted rather than an open research question.

The last row is a separate observation about the upstream semantic extraction rather than about consolidation: **roughly three quarters of semantic triples have an empty object field**, so the "semantic memory" is in practice largely a store of subject–predicate pairs. This is inherited behaviour, not something the consolidation introduced, and it bounds what any consolidation of this store could achieve: it is easier for two subject-predicate pairs to look similar than for two full triples to, which is part of why the cross-subject merge rate is as high as it is.

4.4.5 Node period: a confound that had to be removed

WorldMM builds a semantic node from every ten consecutive chunks. On the 30 s grid that is a five-minute node; at the event condition's 75.9 s mean it would be about thirteen minutes. Comparing a five-minute node against a thirteen-minute node would confound consolidation with node size outright: larger nodes contain more redundancy and would compress better for reasons having nothing to do with the algorithm. **Semantic nodes are therefore built by elapsed time (300 s) in both conditions**, not by chunk count.

A residual asymmetry remains and is disclosed here: **534 nodes in the fixed condition with a mean span of 290 s, against 684 nodes in the event condition with a mean span of 226 s.** Event nodes are about 22% shorter, because a window closes when the next chunk would overshoot it, and longer chunks pack less tightly into a fixed window. The mitigating evidence is Section 4.4.2: compression came out at 1.58x against 1.59x across a 28% difference in node count, so the metric is demonstrably insensitive to node granularity at this scale.

4.5 Database size

Table 4-6 gives the size of every component of the memory under both conditions.

Component	FIXED	EVENT
episodic (OpenIE + triples)	13.11 MB	7.55 MB
semantic extraction	0.82 MB	0.96 MB
consolidated (interval)	1.43 MB	1.67 MB
visual (analytic)	32.14 MB	13.04 MB
total	47.51 MB	23.22 MB
same content in WorldMM format	191.25 MB	236.42 MB
reduction against WorldMM format	4.03x	10.18x

Table 4-6 — Database size by component.

Figure 4-5 shows the same totals broken down by component.

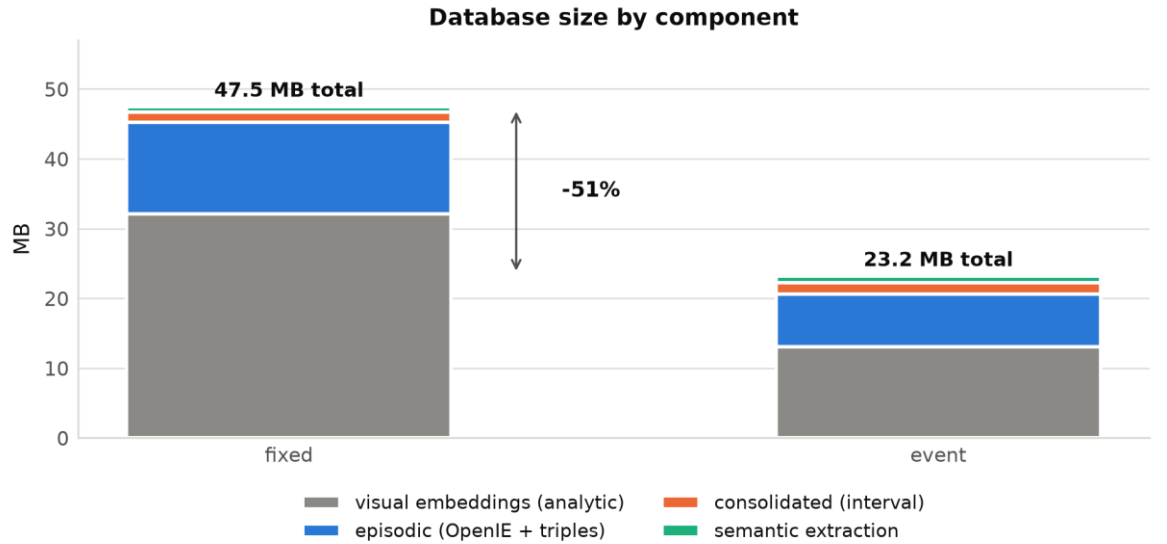


Figure 4-5 — Database size by component. The event condition's database is 51% smaller.

The event condition's database is 51% smaller than the fixed condition's (23.22 MB against 47.51 MB). Two qualifications are essential to reading this correctly.

The mechanism is mostly Objective 1, not Objective 2. Visual embeddings account for 32.14 of the fixed total's 47.51 MB (67.7%) and 13.04 of the event total's 23.22 MB (56.2%). One 1536-dimensional vector is stored per chunk, so 59.4% fewer chunks means 59.4% fewer vectors. The dominant term in the 51% database reduction is therefore a direct consequence of event-boundary indexing, through a different mechanism from the consolidation win in Section 4.4.1. The two are presented separately here for that reason, and should not be added together rhetorically.

Visual sizes are analytic, not measured. For n chunks, each holding one d -dimensional vector of 4-byte floats, with σ a serialisation overhead factor,

$$B_{\text{visual}} = n \cdot d \cdot 4 \cdot \sigma \quad (4.6)$$

with $d = 1536$ and $\sigma = 1.0147$, the latter calibrated against the inherited serialised store (35,367,502 bytes for 5,673 vectors). Equation (4.6) is given so that the figures in the table above can be checked rather than taken on trust. No visual embeddings were regenerated for either condition, because doing so would have consumed GPU time without changing any conclusion: the vector count is exactly the chunk count and the dimensionality is fixed. This is a modelled quantity and is labelled as such in every table.

The episodic component is measured, and falls from 13.11 MB to 7.55 MB. The consolidated component rises slightly, from 1.43 MB to 1.67 MB, because the event condition retains more distinct triples after consolidation.

4.6 Load and indexing latency

4.6.1 Load, model-free

The model-free benchmark loads a semantic database and reconstructs the state at a query time, over 20 query points with 3 repeats.

	FIXED	EVENT
load, WorldMM format	1.28 s	2.10 s
load, interval store	0.22 s	0.19 s
load speedup	5.75x	11.18x
mean reconstructed state	2,715 triples	3,245 triples
per-query reconstruction	7.6 ms	0.83 ms

Table 4-7 — Load latency, model-free (20 query points, 3 repeats).

The interval store's load time is essentially independent of history length, because the file it reads holds one row per distinct fact rather than one copy of the state per node. The reference format's load time grows with the number of nodes, which is why the event condition, with more nodes, is slower to load in that format and therefore shows the larger speedup.

Load timings vary between runs on a shared cluster node according to filesystem cache state: the event condition measured 3.2x, 6.6x and 11.2x across three runs of the same benchmark. A range of **5.7x to 11.2x** is therefore quoted throughout rather than a point estimate. Size and state-size figures are deterministic and do not vary between runs.

4.6.2 Indexing with the embedding model: a negative result

Table 4-8 repeats the measurement with the real embedding model in the loop.

	FIXED	EVENT
WorldMM format	4.35 s (max 8.00)	4.98 s (max 10.27)
interval store	4.16 s (max 8.04)	5.09 s (max 10.24)
speedup	1.05x	0.98x

Table 4-8 — Indexing latency with the real embedding model over 5 query points.

Figure 4-6 places the load result and the indexing result side by side.

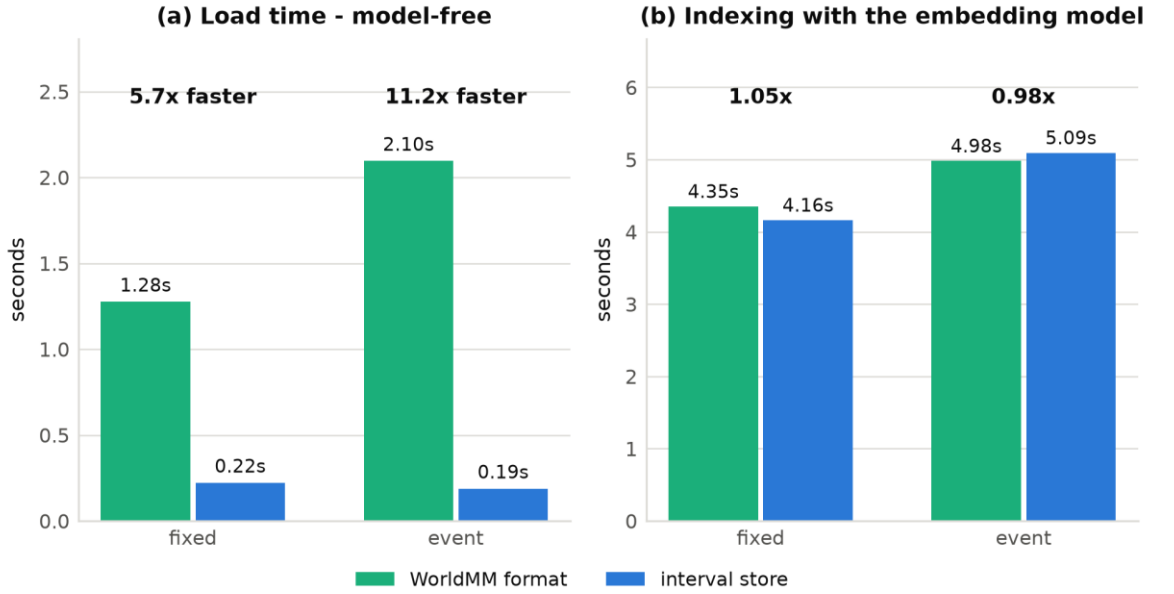


Figure 4-6 — Load time model-free (a) and indexing time with the embedding model (b). The win in (a) does not survive into (b).

There is no indexing-time speedup, and on inspection there could not have been one. The benchmark harness loads both stores *before* the timed region: the interval store is read and materialised into WorldMM's nested representation, and only then is the indexing step timed. Both arms therefore index byte-identical in-memory content, and the timed region contains no part of the operation the interval encoding changes.

Read that way, the result is not a disappointment but a **control**, and it passes. If the interval encoding lost information, if reconstructing the state by filtering on `first_seen` produced a different or smaller state than WorldMM's stored snapshot, the indexing times would differ, because there would be less or different content to embed. They do not differ: 1.05x and 0.98x straddle parity, and the companion measurement of reconstructed state size reports 1.00x. This is consistent with the interval encoding being lossless with respect to the reconstructed state, which is a property the dissertation needs and which no other measurement here addresses.

What the result does rule out is any suggestion that the encoding accelerates the embedding and graph-construction work itself. It does not, and it was never going to: that work is a function of content, and the content is identical by construction.

4.6.3 Cold-start latency: composing the two measurements

Neither benchmark on its own answers the question a system operator would ask, which is how long it takes to go from a cold store on disk to a queryable index. That is load plus index. Composing the two measurements *within the same run*, using the load figures recorded alongside the model-based benchmark rather than those in Table 4-7, so that filesystem state is common, gives the figures in Table 4-9:

	FIXED	EVENT
load, WorldMM format	0.93 s	1.33 s
index, WorldMM format	4.35 s	4.98 s
cold start, WorldMM format	5.28 s	6.31 s
load, interval store	0.16 s	0.41 s
index, interval store	4.16 s	5.09 s
cold start, interval store	4.32 s	5.50 s
end-to-end speedup	1.22x	1.15x

Table 4-9 — Cold-start latency, composed from the load and index benchmarks of the same run.

This is the number to quote for retrieval speed, and it is modest. A 1.15x–1.22x end-to-end improvement is real and reproducible but it is nothing like the 5.7x–11.2x that the load measurement alone suggests. Writing L for load time and X for indexing time, the end-to-end ratio is

$$\text{speedup} = \frac{L_{\text{ref}} + X_{\text{ref}}}{L_{\text{interval}} + X_{\text{interval}}} \quad (4.7)$$

and because X is essentially equal in the two arms while L is only 15 to 21% of the total, equation (4.7) is bounded near 1.2 however fast loading becomes. Section 5.1 uses these figures rather than the load figures when assessing Objective 3, and the load speedup is reported for what it is: a component measurement that shows where the encoding acts, not an end-to-end system result.

The picture changes with scale in the encoding's favour, and the cost model of Section 4.2.1 indicates why: load cost scales with the number of records serialised, which grows quadratically in the node count for the reference format and linearly in distinct facts for the interval store, while indexing cost is a function of content and is the same for both. Over a substantially longer memory, loading would come to dominate cold start for the reference format and not for the interval store. That is an extrapolation, not a measurement, and is offered as such.

4.7 Extraction cost

Table 4-10 gives the measured cost of building each memory.

Stage	FIXED	EVENT
OpenIE, shard 0	11.43 h (16.0 s/chunk)	8.91 h (30.7 s/chunk)
OpenIE, shard 1	11.43 h (16.0 s/chunk)	9.02 h (31.0 s/chunk)
total GPU-hours	22.9	17.9

Table 4-10 — Measured extraction cost, two shards per condition.

Event chunks cost 1.93x per chunk while carrying 2.4x the text: sub-linear, because per-call overhead amortises over longer inputs.

This contradicts an assumption made in the project proposal. The proposal reasoned that because event-boundary chunking produces roughly 60% fewer chunks, and each chunk is one vision-language-model call, the rebuild would be cheaper in roughly the same proportion. Measurement does not support that. **Chunk count fell by 59.4% but GPU cost fell by only 21.8%**, from 22.9 to 17.9 GPU-hours. The reason is that extraction cost is driven by the volume of output tokens rather than by the number of calls, and the total amount of content to be described is conserved however the timeline is cut. Decode time dominates, and decode time scales with what is written. Part of the saving comes from the per-chunk entity extraction that the event condition avoids, and the measurements available here do not separate that part from the rest.

Two supporting measurements. Frame feature extraction, the entire Objective 1 detection frontend, is CPU-only and processed **4,501 videos in 36.5 minutes** with 16 workers and zero failures, which is negligible against 17.9 GPU-hours and confirms the design constraint in Section 3.3.1 was met. Calibration over 50 chunks gives NER at 1.55 s mean, triple extraction at 12.96 s mean, **14.51 s per chunk** overall, at a decode rate of 15.15 tokens/s, with a one-time model load of 2,708 s.

4.8 Retrieval accuracy

The purpose of this evaluation is bounded and should be stated before the numbers. It asks whether event-boundary chunking *damages* retrieval. It is not a benchmark submission and its result is not comparable to published EgoLifeQA scores.

Settings, identical on both arms: 1 retrieval round; 3 episodic candidates; top-2 episodic retrieval; visual retrieval disabled; embedders on CPU; Qwen3-VL-30B as both retriever and responder; 512 maximum new tokens. The retrieval harness is the pre-existing retrieval agent described in Section 1.2.3, used unmodified, so that retrieval policy is held constant and only the memory being retrieved from varies.

The comparison is made over the **38 questions answered under both configurations**, which is

the set on which the two memories can be compared directly. Table 4-11 gives the contingency:

	Count
fixed correct	14 / 38 = 36.8%
event correct	14 / 38 = 36.8%
both correct	9
both wrong	19
fixed only correct	5
event only correct	5
exact McNemar	p = 1.000

Table 4-11 — Matched-pair contingency over the 38 questions both configurations answered.

Figure 4-7 plots the same four outcomes.

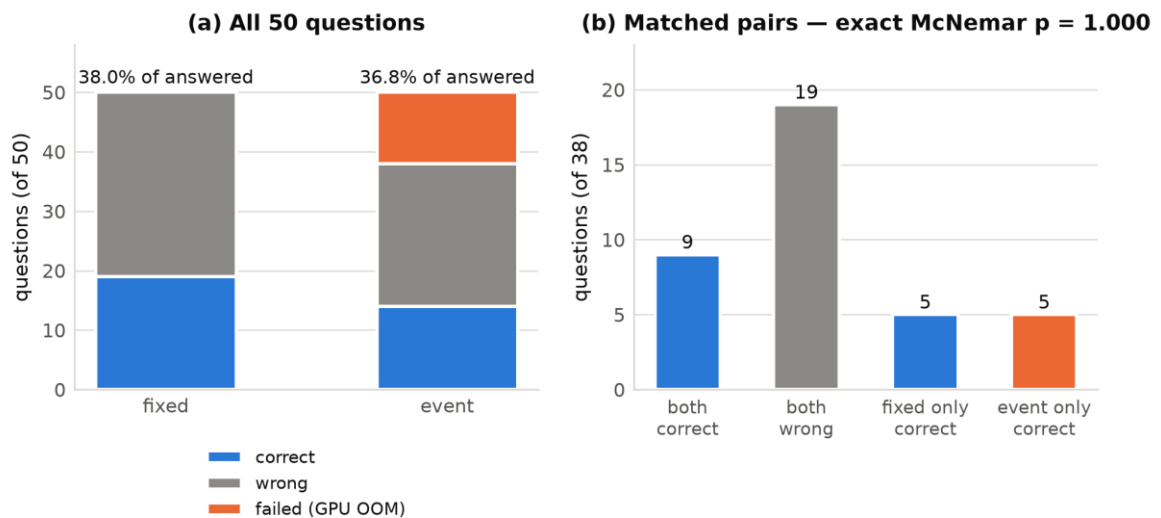


Figure 4-7 — Matched-pair outcomes over the 38 questions answered under both configurations.

Accuracy is identical and the disagreement is symmetric. Each configuration answered five questions correctly that the other got wrong, and McNemar's exact test [30] on the discordant pairs gives $p = 1.000$: the most neutral result the test can return.

4.8.1 How much this evidence is worth

An examiner is entitled to ask what a null result on 38 questions actually rules out, and the honest answer is: very little. Two quantifications make the limit concrete.

The interval on the point estimate is wide. 14 correct of 38 is 36.8%, with an exact Clopper–Pearson 95% confidence interval of **[21.8%, 54.0%]** (Wilson: [23.4%, 52.7%]). Any true accuracy between roughly a fifth and a half of questions is consistent with what was observed, for either arm.

The test could only have detected a total reversal. There are 10 discordant pairs. With 10

discordant pairs, the exact binomial test underlying McNemar reaches $p < 0.05$ only at a **0/10 split** ($p = 0.002$); a 1/9 split gives $p = 0.021$: hence the smallest detectable effect is one where one configuration wins essentially every question the other loses. The observed 5/5 is the least informative outcome possible, and a genuine but moderate difference would have been invisible.

The reading this supports is therefore narrow: **event-boundary chunking does not *obviously* degrade retrieval accuracy** on this subset under these settings, and no difference large enough for this design to detect is present. It does not support a claim of equivalence, and it does not support a comparison with published EgoLifeQA numbers.

On the absolute level of 36.8%. The figure is low, and the reason is the settings rather than the memory. Retrieval was deliberately kept light, at 3 candidates, top-2, one round, with visual retrieval disabled and the embedding models on CPU, so that the whole evaluation fitted inside the available GPU budget. Published EgoLifeQA results use richer retrieval, more rounds and the visual bank. The number here is an internal control for comparing two memories under one policy; it is not a measurement of how well the system answers EgoLifeQA and must not be read as one.

4.8.2 Per-category outcomes

EgoLifeQA labels each question with a reasoning type. The breakdown is reported for completeness, with a caution attached: category sizes are 8 to 14 questions, so no difference here is statistically meaningful and Table 4-12 is descriptive only.

Category	n	Fixed correct	Event correct
EntityLog	8	4	4
TaskMaster	10	3	2
RelationMap	6	0	0
EventRecall	7	2	4
HabitInsight	7	5	4
total	38	14	14

Table 4-12 — Outcomes by EgoLifeQA question category. Descriptive only; cell counts are too small for inference.

One observation, offered as a hypothesis rather than a finding. Both conditions are weakest on RelationMap, the category that requires facts about how people relate to one another, and on which neither answered any of its six questions correctly. That is the same category the cross-subject merge errors of Section 4.4.4 would be expected to damage. Confirming the connection would need a larger question set.

4.9 Threats to validity

1. **Single subject.** One EgoLife participant, 43 recorded hours. Nothing here establishes that the boundary statistics generalise to the other five EgoLife participants, whose activity patterns differ.
2. **Raw triple counts are not comparable across conditions.** The event condition extracted 43,888 triples against the fixed condition's 75,399, but only 10.3% of the fixed condition's unique triples appear in the event condition's set, and entity overlap is 49.9%. Triples are unnormalised free text, so the same fact expressed over a different grouping of the source material becomes a different string. **Any compression ratio built on raw triple counts across conditions compares incommensurable quantities**, and no such ratio is reported in this dissertation. Compression is measured only post-consolidation, where canonicalisation makes the sets comparable.
3. **Triple yield scales sub-linearly with chunk length:** 2.4x the text yields 1.44x the triples per chunk. This is partly genuine abstraction over longer spans and partly the model's prior towards producing a fixed-size answer regardless of input length. The two cannot be separated with the data available.
4. **Visual database sizes are analytic, not measured** (Section 4.5).
5. **No index-time speedup** from the interval encoding (Section 4.6.2). The win is storage and load.
6. **The retrieval evaluation is bounded:** 38 matched questions, one retrieval round, text-only memory (Section 4.5).
7. **Semantic node spans differ by 22% between conditions** (Section 4.4.4), though the compression metric is shown to be insensitive to this.
8. **The empty-chunk rate is asymmetric:** 3.63% for fixed against 7.80% for event, a 2.15x gap that persisted after a targeted retry pass. Longer inputs fail structured parsing more often, which is a cost of Objective 1 that partially offsets its benefit and which the hardened parser of Section 3.2 reduces but does not eliminate.
9. **Coverage differs between the two conditions** for the structural reason given in Section 3.1.3, so chunk and triple counts rather than aggregate duration are the comparable quantities.
10. **Consolidation merges by embedding similarity only.** A language model could merge paraphrases that share no embedding neighbourhood, and could generate a better-worded merged triple than any existing cluster member. The threshold sweep partly exposes this limit: a threshold low enough to catch genuine paraphrases (0.78) also merges distinct facts, which is why the operating point sits at 0.86 and accepts less compression.
11. **About one merge in nine crosses subjects.** 11.0% of absorbed variants in the fixed

condition and 14.3% in the event condition have a different subject from the representative that absorbed them (Section 4.4.4). Some are legitimate reciprocals, but some conflate facts about different people. This is the most serious correctness limitation in the work and it is not mitigated in the submitted system.

12. **The semantic store is largely subject–predicate pairs.** Around three quarters of consolidated triples carry an empty object field (74.2% fixed, 72.7% event). This is inherited from the upstream semantic extraction rather than introduced here, but it weakens every downstream claim about "triples" and contributes to the cross-subject merge rate.
13. **Visual fidelity was never evaluated.** Event chunking stores one 1536-dimensional vector per chunk, so it stores 2,092 vectors where the baseline stores 5,156: which is where most of the 51% database reduction comes from (Section 4.5). One embedding now summarises up to 300 s of video instead of 30 s, which must lose visual detail. **Because visual retrieval was disabled in the evaluation (Section 4.8), that loss was never measured.** The storage saving is therefore certain and the capability cost is unquantified; a reader should treat the visual component of the 51% figure as a reduction whose retrieval consequences are unknown rather than as a free saving. This is the largest unmeasured risk in the dissertation.
14. **Extraction is not fully deterministic.** The hardened parser of Section 3.2 escalates to sampled generation on retry, so the 473 calls that failed to parse at the first attempt were re-run with sampling enabled and are not reproducible byte for byte. This affects roughly 3% of calls in each condition; the first attempt, which covers the other 97%, remains greedy and deterministic.
15. **Load timings are from a shared cluster node** and vary with filesystem cache state (Section 4.6.1). Ranges are quoted rather than point estimates wherever this applies.

4.10 Summary

Objective 2 is met: an offline consolidation pass that merges near-duplicate semantic triples by embedding similarity and stores each fact once with a validity interval, reducing consolidated storage by 101.4x and 128.4x on identical content while issuing no language-model calls at all. The method is not uniformly sound, however: Section 4.4.4's audit finds that 11.0%–14.3% of absorbed variants carry a different subject from the row that absorbed them, which is a correctness defect of the submitted system and is reported as one.

Objective 3 is met in the sense that the impact was measured across compression, database size, latency, extraction cost and retrieval accuracy: with the measurement returning a mixed verdict that is reported as such. The database is 51% smaller, though most of that is fewer visual vectors whose retrieval cost was never measured. The consolidated store loads 5.7x–11.2x faster, which composes to 1.22x and 1.15x at cold start; there is no index-time speedup, and Section 4.6.2 shows the benchmark could not have produced one. Consolidation does not compound with chunking. Extraction is

21.8% cheaper rather than proportionally cheaper. Retrieval accuracy is unchanged on the questions both systems answered, on evidence weak enough that only a near-total reversal would have been detectable.

5 CONCLUSION

This dissertation asked what changes when a long-form video memory system stops dividing its input by the clock and stops re-writing its semantic state at every update. Two mechanisms were built against the WorldMM reference pipeline and measured on 43 hours of egocentric video from EgoLife subject A1_JAKE, under a controlled ablation in which the captioner, prompts, embedding models and all downstream code were held constant and only the segmentation varied.

Chapter 3 presented the event-boundary indexing module: HSV colour histograms and grayscale thumbnails sampled at 2 fps, differenced between adjacent frames and scored against a rolling median and median absolute deviation, with the operating threshold chosen on the share of boundaries that detection rather than the length cap places. It also documented an engineering finding that had to be resolved before any comparison was valid: a silent parser failure that had suppressed 13.2% of the inherited baseline.

Chapter 4 presented the interval-encoded semantic store and the sleep-cycle consolidation algorithm, and carried the full evaluation: storage, compression, database size, load and index latency, extraction cost, and retrieval accuracy on EgoLifeQA. It reported the results that support the approach and the three that do not.

5.1 Evaluation

The objectives in Section 1.2 are assessed against measured outcomes, summarised in Table 5-1 and discussed individually below.

Objective	Outcome	Assessment
1. Event-boundary aware indexing that segments on semantic state change rather than fixed windows	Chunk count 5,156 $\rightarrow$ 2,092 (−59.4%); mean length 27.9 s $\rightarrow$ 75.9 s; 75.2% of boundaries from detection; 75.6% of chunks span more than one source file; no VLM in the detection loop; front-end cost 36.5 min on CPU	Met
2. Offline "sleep cycle" consolidation collapsing repetitive episodic material into compressed semantic nodes	Interval store 1.43 MB / 1.67 MB against 145.18 MB / 214.88 MB for identical content (101.4x / 128.4x); 0 language-model calls against ~8,757 / ~10,347; 1.58x / 1.59x semantic compression at threshold 0.86	Met
3. Evaluate the impact on database	Database 51% smaller (23.22 MB vs 47.51 MB); load 5.7x–11.2x faster; no index-time speedup (1.05x / 0.98x);	Met, with a partly

Objective	Outcome	Assessment
compression ratios and retrieval speed	extraction 21.8% cheaper rather than proportionally cheaper; retrieval accuracy identical (36.8% both, McNemar $p = 1.000$)	negative result

Table 5-1 — Objectives against measured outcomes.

Objective 1 is met, and met in the strong sense. The claim that segmentation is driven by content rather than by structural constraints is not asserted but measured: every cut carries its origin, and 75.2% of them come from detection. The result is stable across days (70%–82%). The threshold was selected from a measured curve rather than tuned against a downstream metric, which preserves the independence of the retrieval evaluation. The front-end runs on CPU in 36.5 minutes, so it is free relative to the 17.9 GPU-hours of extraction it feeds.

The objective's own phrasing deserves one qualification. It says "semantic state changes", and the implemented detector responds to *visual* change, which is a proxy. There are no human boundary annotations for EgoLife against which the proxy could be validated directly, so the evidence that it is a good proxy is indirect: the boundaries produce chunks that do not degrade retrieval (Section 4.8) while covering the same material in 59.4% fewer units. That is real evidence but it is weaker than a direct boundary-quality measurement would be, and Section 5.2 treats closing that gap as the first priority.

Objective 2 is met. The consolidation algorithm collapses repetitive material and does so at a cost the reference implementation does not approach, 101.4x and 128.4x less storage for identical content, with zero generative calls. The "sleep cycle" framing is honoured structurally: an offline pass, run in cycles to a fixed point, that generalises across repeated observations while preserving their aggregate contribution as support counts. The analogy is a design motivation, and the dissertation does not claim mechanistic equivalence with biological consolidation.

One deviation from the objective's wording must be declared. Objective 2 says "collapse highly repetitive episodic *event boundaries* into singular, compressed *semantic nodes*". The implemented algorithm collapses the semantic *triples* derived from those boundaries, and it does not merge nodes at all: nodes remain on a fixed 300 s grid (Section 4.4.5). The reason is a property of the reference system discovered during implementation and recorded in Section 2.2: nothing in the pipeline stores free-text captions, only triples, so the "re-aggregate and summarise the source text" reading of the objective was not available. Re-aggregation had to mean merging and de-duplicating triple sets. The substance of the objective, an offline pass that collapses repetition into compressed semantic structure: is delivered; the unit on which it operates is not the one the proposal anticipated, and the objective would be more accurately phrased as "...collapse highly repetitive semantic content derived from episodic event boundaries...".

Three further limits should be recorded. The compression achieved on the triple set itself is modest, 1.58x at the operating point, because a threshold aggressive enough to catch paraphrases also destroys distinct facts, and the operating point deliberately sacrifices compression for fidelity. The merge is by embedding similarity only, so paraphrases with no embedding overlap are missed, and the retained phrasing is the best existing member rather than the best possible phrasing. And, most seriously, **about one absorbed variant in nine has a different subject from the representative that absorbed it** (11.0% and 14.3%, Section 4.4.4): the whole-triple embedding that lets the method see past predicate phrasing can also let it merge past a subject mismatch. That is a correctness defect in the submitted system, not merely a missed opportunity, and Section 5.2 gives the fix.

Objective 3 is met, and the measurement is partly unfavourable. The objective asked for an evaluation of impact on "database compression ratios and retrieval speed", and the evaluation delivered on both, with results in both directions:

- Database size falls 51%, and the total against the reference format falls 4.03x and 10.18x. But most of the 51% is fewer visual vectors, whose retrieval cost was never measured because visual retrieval was disabled (Section 4.9, item 13).
- Consolidated storage falls 101.4x and 128.4x on identical content, and the cost model of Section 4.2.1 reproduces both figures exactly from independently measured terms.
- **Cold-start latency, the figure that answers "retrieval speed", improves 1.22x (fixed) and 1.15x (event).** The 5.7x–11.2x load speedup is a component measurement; loading is only 15–21% of cold start, so Amdahl's law bounds the end-to-end gain near what was observed.
- **Index time does not improve** (1.05x, 0.98x), and Section 4.6.2 establishes that it could not have: the benchmark loads both stores before timing, so parity is the expected outcome and its value is as a control confirming the encoding is lossless.
- Retrieval accuracy is unchanged, but the evidence is weak in a way worth stating: with 10 discordant pairs the test could only have detected a near-total reversal, and the 95% interval on 14/38 runs from 21.8% to 54.0%.

Three of the project's original expectations were contradicted by measurement, and volunteering them is more useful than defending them. The proposal assumed event chunking would be substantially cheaper to build; it is 21.8% cheaper, not proportionally cheaper, because extraction cost tracks output volume rather than call count. It assumed consolidation would compound with event chunking; the two compress identically (1.58x against 1.59x) and are orthogonal. It did not anticipate that longer chunks would inflate inference-time memory, which they do, one-directionally and by an amount that matters. And the working assumption that a lighter store would index faster was wrong twice over: the index-time benchmark cannot show such an effect by construction, and the end-to-end gain, once measured properly, is 1.15x–1.22x rather than the order of magnitude the load figure

alone implies.

5.2 Reflection on the conduct of the project

The template asks for evidence that the project was tackled in a professional manner. Four decisions carry that weight, and each cost something.

Both arms were rebuilt rather than one reused. The inherited memory was available and would have served as a baseline at no compute cost. It was discarded and both conditions rebuilt from the raw stream, roughly 41 GPU-hours on a contended cluster, because it carried an extraction defect that would otherwise have been measured as an effect of chunking. This was the single most expensive decision in the project and the one that most directly determines whether its central comparison means anything.

The defect was fixed even though fixing it raised the bar. Correcting the extraction path lifted the baseline from 72,141 to 75,399 triples on identical chunks, which makes every comparison in Chapters 3 and 4 harder to win. Leaving it in place would have flattered the result.

The central claim was made falsifiable by construction. Tagging every boundary with the rule that produced it means the assertion that detection drives the segmentation can be checked rather than taken on trust, and would have been visibly false had the length cap been doing the work. The detection threshold was likewise fixed before any retrieval was run, and on a property of the segmentation rather than of the evaluation, so it could not be tuned against the metric used to judge it.

Negative results were sought and reported. The absence of an indexing-time speedup, the independence of the two contributions, the cost saving that fell well short of what the proposal assumed, and the cross-subject merge rate all run against the thesis and are in the main text rather than in a footnote. The merge-quality audit in particular was run because compression ratios alone cannot show whether a consolidation method is correct, and it found a defect that the compression figures conceal.

Two scope decisions should also be recorded, since both bound what the dissertation can claim. Visual retrieval was disabled throughout the evaluation so that the whole question set could be run inside the available GPU budget; the consequence is the unmeasured risk at item 13 of Section 4.9. And the work covers one participant rather than six, which is a compute decision rather than a methodological one, and is the first thing Section 5.3 would spend more resources on.

Supporting this, the pipeline is checkpointed and idempotent at every stage, so an interrupted run resumes rather than restarting; a suite of 155 test functions accompanies the implementation; and Appendix B records the configuration in enough detail for the experiment to be repeated.

5.3 Future Work

Validate the boundary proxy directly. The single clearest gap is that visual change is used as a proxy for event change with no direct measurement of how good a proxy it is. A modest human annotation exercise, a few hours of A1_JAKE segmented independently by two or three annotators, would allow the detector to be scored with the standard GEBD F1 protocol [23] and would convert the strongest claim in Chapter 3 from indirect to direct evidence. This is cheap and should be done first.

Learned or multimodal boundary detection. With annotations available, a learned detector becomes evaluable, as does the obvious multimodal extension: the audio track and the transcript both carry boundary evidence that the current detector ignores entirely. A speaker change or a several-second silence is often a stronger event cue than any visual change, and the transcript is already in the pipeline at no extra cost.

Adaptive rather than capped maximum length. 23.1% of boundaries currently come from the 300 s cap, which is a blunt instrument. A detector that lowered its threshold as time since the last boundary grew would convert most cap cuts into detected ones and would make the length distribution less dependent on an arbitrary constant.

Constrain merging by subject. This is the highest-value single change available, and it is small. Section 4.4.4 measures that 11.0%–14.3% of absorbed variants have a different subject from their representative, because whole-triple embedding lets a shared predicate phrase outweigh a subject mismatch. Two remedies are immediate: partition the clustering by normalised subject so that merges can only occur within a subject, or embed subject and predicate separately and require both to pass a threshold. Either would be a few hours of work and would make the store's correctness defensible; neither was implemented here, and the cross-subject rate should be re-measured after the change as the primary success criterion.

Measure what the visual bank loses. Section 4.9 item 13 identifies the largest unmeasured risk in this dissertation: event chunking stores 2,092 visual vectors where the baseline stores 5,156, which is most of the 51% database saving, and one embedding now summarises up to 300 s of video. Re-running the evaluation with visual retrieval *enabled* on both arms would convert that from an unknown into a measurement, and would settle whether the storage saving is free or paid for in visual recall. This should be done before the storage result is presented anywhere as a net win.

Bound the retrieval context rather than the chunk. Event chunks carry 2.4 times the text of fixed chunks on average, and the upper tail is far heavier still, with p95 text length rising from 954 to 5,026 characters (Table 3-7). Whatever is retrieved, the longest chunks therefore place a great deal of text into the prompt, and the cost of every generated answer scales with it. The remedy is not to shorten the chunks but to retrieve sub-spans of the long ones: keep the event as the unit of indexing,

and select a bounded window within it for prompt assembly. This would preserve the storage saving while bounding what any single chunk can contribute to a prompt, and it is the natural next experiment.

Hybrid consolidation. Section 4.9 identifies the limit of embedding-only merging. A hybrid that uses embedding similarity to *propose* clusters and a language model only to *adjudicate* the ambiguous band, say, cosine similarity between 0.78 and 0.86, would recover paraphrases that the current method misses at a small fraction of WorldMM's per-triple cost. The threshold sweep in Table 4-3 already identifies exactly which band that is.

Continuous rather than batch consolidation. The current pass is offline over the whole store. A system running indefinitely would need consolidation to run incrementally, on a schedule, over a recent window: which is closer to the biological analogy than the batch implementation is, and which raises a question this project did not address: whether repeatedly consolidating already-consolidated material degrades it over time.

A retention policy built on the intervals. Section 1.2.4 notes that cheap memory is memory that gets kept, and that the interval encoding happens to carry exactly the metadata a retention policy needs: every fact knows when it was first and last observed. A store that expires facts whose `last_seen` falls outside a retention window, or that supports selective forgetting of a named entity across the whole history, is a small extension of the data structure and a substantive contribution in its own right. It is listed here as future work rather than claimed as a property of what was built.

Model scale. The vision-language model was deliberately held constant across both arms, at a single 30B checkpoint, so that the ablation isolates the chunking and storage changes rather than confounding them with model capability. The consequence is that nothing here establishes how far the results depend on that choice. A sweep across model sizes, run under the same parity conditions as Section 3.1.3 requires, would answer two separate questions: whether the extraction quality that consolidation depends on survives at smaller scales, and whether the storage and latency results, which are properties of the encoding rather than of the model, are invariant to it as they should be. The second is the cheaper and more informative of the two.

Scale and generality. All results are from one subject. Repeating the pipeline across all six EgoLife participants would establish whether the 75.2% detection share and the 59.4% chunk reduction are properties of the method or of one person's week, and would give the retrieval evaluation the sample size Section 4.8.1 shows it currently lacks: six subjects would put roughly six times as many discordant pairs into the McNemar test, in place of the ten available here.

REFERENCES

- [1] Yeo, W., Kim, K., Yoon, J. and Hwang, S. J. WorldMM: Dynamic Multimodal Memory Agent for Long Video Reasoning. In Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2026. (Highlight.) arXiv:2512.02425.
- [2] Yang, J. et al. EgoLife: Towards Egocentric Life Assistant. In Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2025. arXiv:2503.03803.
- [3] Zacks, J. M., Speer, N. K., Swallow, K. M., Braver, T. S. and Reynolds, J. R. Event perception: A mind-brain perspective. *Psychological Bulletin*, 133(2):273–293, 2007.
- [4] McGaugh, J. L. Memory: a century of consolidation. *Science*, 287(5451):248–251, January 2000.
- [5] McClelland, J. L., McNaughton, B. L. and O'Reilly, R. C. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457, 1995.
- [6] Kumaran, D., Hassabis, D. and McClelland, J. L. What learning systems do intelligent agents need? Complementary learning systems theory updated. *Trends in Cognitive Sciences*, 20(7):512–534, July 2016.
- [7] Gutiérrez, B. J., Shu, Y., Gu, Y., Yasunaga, M. and Su, Y. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [8] Liu, H., Li, C., Wu, Q. and Lee, Y. J. Visual instruction tuning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [9] Qwen Team. Qwen3-VL Technical Report. arXiv:2511.21631, 2025. (Checkpoint used in this work: Qwen/Qwen3-VL-30B-A3B-Instruct.)
- [10] Lewis, P. et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 9459–9474, 2020.
- [11] Song, E. et al. MovieChat: From dense token to sparse memory for long video understanding. In Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2024.
- [12] He, B. et al. MA-LMM: Memory-augmented large multimodal model for long-term video understanding. In Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2024.
- [13] Wang, X., Zhang, Y., Zohar, O. and Yeung-Levy, S. VideoAgent: Long-form video understanding with large language model as agent. In Proc. European Conf. Computer Vision (ECCV), 2024.
- [14] Packer, C. et al. MemGPT: Towards LLMs as operating systems. arXiv:2310.08560, 2023.
- [15] Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P. and Bernstein, M. S. Generative agents: Interactive simulacra of human behavior. In Proc. ACM Symp. User Interface Software

- and Technology (UIST), 2023.
- [16] Jiang, Z., Meng, R., Yang, X., Yavuz, S., Zhou, Y. and Chen, W. VLM2Vec: Training vision-language models for massive multimodal embedding tasks. In Proc. Int. Conf. Learning Representations (ICLR), 2025. arXiv:2410.05160.
 - [17] Banko, M., Cafarella, M. J., Soderland, S., Broadhead, M. and Etzioni, O. Open information extraction from the web. In Proc. Int. Joint Conf. Artificial Intelligence (IJCAI), pages 2670–2676, 2007.
 - [18] Angeli, G., Premkumar, M. J. J. and Manning, C. D. Leveraging linguistic structure for open domain information extraction. In Proc. Annual Meeting of the Association for Computational Linguistics (ACL), pages 344–354, 2015.
 - [19] Lienhart, R. Comparison of automatic shot boundary detection algorithms. In Proc. SPIE Storage and Retrieval for Image and Video Databases VII, volume 3656, pages 290–301, December 1998.
 - [20] Smeaton, A. F., Over, P. and Doherty, A. R. Video shot boundary detection: Seven years of TRECVID activity. *Computer Vision and Image Understanding*, 114(4):411–418, April 2010.
 - [21] Castellano, B. PySceneDetect: Video scene cut detection and analysis tool. <https://www.scene-detect.com>, accessed August 2026.
 - [22] Rasheed, Z. and Shah, M. Detection and representation of scenes in videos. *IEEE Transactions on Multimedia*, 7(6):1097–1105, December 2005.
 - [23] Shou, M. Z. et al. Generic event boundary detection: A benchmark for event segmentation. In Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV), pages 8075–8084, 2021.
 - [24] Wilson, M. A. and McNaughton, B. L. Reactivation of hippocampal ensemble memories during sleep. *Science*, 265(5172):676–679, July 1994.
 - [25] Diekelmann, S. and Born, J. The memory function of sleep. *Nature Reviews Neuroscience*, 11(2):114–126, February 2010.
 - [26] Tulving, E. Episodic and semantic memory. In Tulving, E. and Donaldson, W., editors, *Organization of Memory*, pages 381–403, Academic Press, New York, 1972.
 - [27] Rebuffi, S.-A., Kolesnikov, A., Sperl, G. and Lampert, C. H. iCaRL: Incremental classifier and representation learning. In Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), pages 2001–2010, 2017.
 - [28] Grauman, K. et al. Ego4D: Around the world in 3,000 hours of egocentric video. In Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), pages 18973–18990, 2022. arXiv:2110.07058.
 - [29] Leys, C., Ley, C., Klein, O., Bernard, P. and Licata, L. Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median. *Journal of Experimental Social Psychology*, 49(4):764–766, July 2013.
 - [30] McNemar, Q. Note on the sampling error of the difference between correlated proportions or

percentages. *Psychometrika*, 12(2):153–157, June 1947.

- [31] Meng, R., Jiang, Z., Liu, Y., Su, M., Yang, X., Fu, Y., Qin, C., Chen, Z., Xu, R., Xiong, C., Zhou, Y., Chen, W. and Yavuz, S. VLM2Vec-V2: Advancing multimodal embedding for videos, images, and visual documents. *Transactions on Machine Learning Research (TMLR)*, 2026.

APPENDIX A — ORIGIN OF THE COMPONENTS USED

Component	Origin
Frame-feature extraction and boundary detection	Built for this dissertation
Chunk assembly and the content-addressed identifier scheme	Built for this dissertation
Hardened episodic extraction path (Section 3.2)	Built for this dissertation
Time-based semantic node construction	Built for this dissertation
Interval-encoded semantic store and consolidation algorithm	Built for this dissertation
Metrics harness behind every figure and table	Built for this dissertation
Chunk, memory, consolidation, metric and evaluation outputs for both conditions	Produced by the above
One earlier run of the reference pipeline over the same subject	Inherited from the shared codebase. Used in Section 4.1 as a behavioural reference only, never as the baseline arm
Retrieval agent and evaluation harness (Section 4.8)	Not written by the author. Used exactly as received
Spatial memory bank	Not written by the author. Not exercised by any experiment reported here
EgoLife video, captions and transcripts	Upstream dataset release
Reference system source	Upstream release

Both experimental conditions were rebuilt from the raw stream rather than reusing the inherited artifacts, because the inherited episodic memory was degraded by the extraction defect documented in Section 3.2.

APPENDIX B — EXPERIMENTAL CONFIGURATION

B.1 PIPELINE STAGES AND THEIR COST

Stage	What it does	Hard-ware	Cost
1. Frame features	HSV histograms and grayscale thumbnails at 2 fps over the full corpus	16 CPU cores	36.5 min
2. Boundary detection and chunk assembly	Rolling median/MAD scoring, boundary-to-chunk rules, text aggregation	CPU	seconds
3. Episodic extraction	Entity and triple extraction per chunk, two shards per condition	2 x A100-80GB	22.9 GPU-h fixed, 17.9 GPU-h event
4. Semantic extraction	300 s time-based nodes over the episodic stream	1 GPU	minutes
5. Consolidation	Five-point similarity sweep per condition	1 GPU	513 s fixed, 594 s event
6. Metrics	Sizes, latencies and figure data, with and without the embedding model	CPU and 1 MIG slice	minutes
7. Evaluation	50 EgoLifeQA questions per condition under identical settings	1 A100-80GB	hours

Stages 3 to 7 are checkpointed per chunk and idempotent, so an interrupted run resumes rather than restarting.

B.2 CONSOLIDATION CONFIGURATION

Cosine similarity threshold 0.86; cycles repeated to a fixed point; representative selected as the highest-support member of each cluster, ties broken towards the shortest surface form; text embeddings from Qwen3-Embedding-4B.

B.3 EVALUATION CONFIGURATION

One retrieval round; three episodic candidates; top-2 episodic retrieval; visual retrieval disabled; embedding models on CPU; Qwen3-VL-30B-A3B-Instruct as both retriever and responder; 512 maximum new tokens. Identical on both arms.

B.4 ENVIRONMENT

University of Surrey Eureka. Two NVIDIA A100-80GB GPUs on a contended partition, MIG slices on a second node for smaller jobs, and a shared CPU partition for feature extraction. Models: Qwen3-VL-30B-A3B-Instruct for captioning, extraction and question answering; Qwen3-Embedding-4B for text embeddings; VLM2Vec-V2.0 for visual embeddings. Model load takes 32 to 45 minutes depending on filesystem cache state.

Following the template's guidance, no program listings appear in these appendices. The complete source, configuration files and test suite are submitted separately as a repository.

APPENDIX C — A WORKED EXAMPLE: ONE SPAN THROUGH THE WHOLE PIPELINE

The chapters describe each stage separately. This appendix traces a single span of video through all of them, so that the shape of the data at each point is concrete. Every value below is taken from the recorded outputs of the fixed condition; nothing is illustrative.

C.1 Stage 1 — the source span

The span begins at Day 1, 11:09:58 and runs for 15 seconds. Its aggregated caption and transcript text, which is what the memory is built from, reads:

I'm holding my phone. I am holding my phone while facing the table. I said a sentence. Okay, then we need a stopwatch. I hold the phone in both hands. I'm sliding my phone. I start operating the smartphone. Shure: Mark a timestamp, right? Everyone on the table is looking at me. Yes, mark it. I am continuing to operate the phone.

This is 330 characters, close to the median of 551 for the fixed condition (Table 3-7). It illustrates the redundancy the two contributions target: the same fact, that the wearer is using a phone, is restated five times in fifteen seconds.

C.2 Stage 2 — episodic extraction

The vision-language model returns nine triples in Table C-1 for this span:

Subject	Relation	Object
I	am holding	my phone
I	hold	the phone in both hands
I	start operating	the smartphone
I	sliding	my phone
everyone on the table	looking at	me
Shure	marks	a timestamp
I	continue operating	the phone
my phone	is same as	the smartphone
the phone	used by	I

Table C-1 — Episodic triples extracted from the span.

Four of the nine describe the same activity with four different relation phrases. None of them is a string match for any other, which is the unnormalised-vocabulary problem of Section 4.1.1 appearing at its source. Note also that a span this short and this repetitive is exactly what a fixed 30-second grid produces in quantity, and what event-boundary indexing aggregates into a longer unit.

C.3 Stage 3 — semantic node

The first semantic node of the run closes at Day 1, 11:14:30, covering the preceding 300 seconds of elapsed time and therefore this span among others. It holds five triples in Table C-2:

Subject	Relation	Object
I	uses	smartphone
I	owns	phone
I	operates	phone
I	interacts with	others through communication devices
I	controls	technology related to computing environments

Table C-2 — Semantic triples at the first node.

The abstraction from episodic to semantic is visible: specific acts ("hold the phone in both hands") have become dispositions ("uses", "owns", "operates"). So is the problem: the node has produced three near-synonyms for one fact, and under the reference system's accumulate-and-rewrite scheme all three would persist and be re-serialised at every subsequent node for the remainder of the week.

C.4 Stage 4 — consolidation

Consolidation places all three of those into one cluster, along with paraphrases that arrive later in the week. The retained row is given in Table C-3:

Field	Value
triple	(I, uses, phone)
first seen	Day 1, 11:14:30
last seen	Day 7, 18:34:56
support	34
variants absorbed	8

Table C-3 — The consolidated interval record.

The eight absorbed variants are *(I, owns, phone)*, *(I, can operate, phone)*, *(I, often uses, phone)*, *(I, interacts frequently with, phone)*, *(I, uses, Phone)*, *(I, uses_phone,)*, *(I, uses, mobile phone)* and *(I, can use, phone)*.

This single row is the whole argument of Chapter 4 in miniature. Thirty-four separate observations spanning six days of recording are held as one record with a validity interval, rather than as thirty-four strings re-written at every node between their first appearance and the end of the week. It also shows the method working on exactly the failure it was designed for: *(I, owns, phone)* and *(I, uses, phone)* differ in every character of their relation and could never merge by string equality, but sit close together once the whole triple is embedded.

Two blemishes visible in the same row are worth noting, because they are the ones Section 4.4.4 quantifies. *(I, uses, Phone)* differs from the representative only in capitalisation and should have been caught by normalisation before embedding was needed at all. And *(I, uses_phone,)* is a malformed extraction with an empty object, of the kind that accounts for roughly three quarters of the consolidated store.

APPENDIX D — GLOSSARY AND NOTATION

Terms are defined as this dissertation uses them. Where a term has a broader meaning in the literature, the narrower sense intended here is given.

Term	Meaning in this dissertation
Chunk	The unit of memory: a contiguous span of video, together with the caption and transcript text falling inside it. Fixed chunks are 30-second windows; event chunks run between detected boundaries
Boundary	A time at which one chunk ends and the next begins. Tagged by the rule that produced it: <i>detected, cap or gap</i>
Detected boundary	A boundary placed because the frame-to-frame visual dissimilarity exceeded the adaptive threshold
Cap boundary	A boundary forced because no detection had occurred for 300 seconds
Gap boundary	A boundary forced by an interval of unobserved video
Episodic memory	The store of what happened: subject–relation–object triples extracted per chunk
Semantic memory	The store of generalised knowledge, built by periodic aggregation over the episodic stream
Visual memory	One dense embedding vector per chunk, supporting visual similarity retrieval
Semantic node	A periodic checkpoint of the semantic store. Built here from 300 seconds of elapsed time rather than from a fixed number of chunks
Triple	A fact of the form (subject, relation, object), produced by open information extraction. Relations are free text and are not drawn from a controlled vocabulary
OpenIE	Open information extraction: turning free text into triples without a predefined schema
Support	The number of observations a consolidated triple aggregates
Variant	A triple absorbed into another during consolidation, retained so that the merge can be audited
Interval store	The encoding introduced here: each distinct fact written once with first-seen and last-seen timestamps, with the state at any time reconstructed by filtering
Reference format	The encoding used by the baseline system: the complete accumulated state re-written at every semantic node
Consolidation	The offline pass that merges near-duplicate semantic triples and records them as intervals
Cycle	One pass of embed, cluster and merge within consolidation, repeated to a fixed point

Term	Meaning in this dissertation
Cold start	Loading a store from disk and building a queryable index from it
Matched set	The questions answered under both configurations, on which the two memories are compared

Symbol	Meaning	Introduced
d_i	Visual dissimilarity between adjacent sampled frames	Equation (3.1)
$\mathbf{h}_i, \mathbf{g}_i$	HSV colour histogram and grayscale thumbnail of frame i	Section 3.3.2
w	Weight balancing colour against layout, set to 0.5	Equation (3.1)
w	Rolling window over which the threshold is estimated, 60 s	Equation (3.2)
MAD_i	Median absolute deviation of the dissimilarity within W	Equation (3.2)
z_i	Robust standard score of the dissimilarity	Equation (3.3)
τ	Detection threshold on z_i , set to 4.0	Section 3.3.3
δ	Absolute floor on d_i , set to 0.02	Section 3.3.3
I	The set of facts held in the interval store	Equation (4.1)
$S(t)$	The state reconstructed for time t	Equation (4.1)
N	Number of semantic nodes	Section 4.2.1
s_n	Triples held in the state at node n	Equation (4.2)
F	Distinct facts in the final store	Equation (4.3)
$R_{\text{ref}}, R_{\text{interval}}$	Triple-records serialised under each encoding	Equation (4.2), (4.3)

Symbol	Meaning	Introduced
$b_{\text{ref}}, b_{\text{interval}}$	Mean bytes per record under each encoding	Equation (4.4)
c	Compression, as observations per retained fact	Equation (4.5)
d, σ	Embedding width and serialisation overhead	Equation (4.6)
L, X	Load time and indexing time	Equation (4.7)